Kylin
First-Stage NGCC Digital-Signature Submission Package

Kylin Team

April 27, 2026

Contents

1 Algorithm Description **4**

2 Main Design Differences from SPHINCS$^+$ **5**
2.1 Diverse Layer Heights in the Hypertree . . . 5
2.2 The WOTS$^+$ Encoding . . . 5
2.3 Size-Optimal Encoding . . . 5
2.4 Theoretical Performance . . . 8

3 Notation and Supporting Definitions **9**
3.1 Data Types . . . 9
3.2 Functions . . . 9
3.3 Operations . . . 9
3.4 Integer to Byte Conversion (Function `toByte`) . . . 10
3.5 Member Functions (Functions `set, get`) . . . 10
3.6 Cryptographic (Hash) Function Families . . . 10
3.6.1 Tweakable Hash Functions (Functions `T, F, H`) . . . 10
3.6.2 PRF and Message Digest (Functions `PRF, PRF_msg, H_msg`) . . . 10
3.6.3 Hash Function Address Scheme (Structure of `ADRS`) . . . 10

4 CS-WOTS$^+$ One-Time Signatures **12**
4.1 CS-WOTS$^+$ Parameters . . . 12
4.2 CS-WOTS$^+$ Chaining Function (Function `chain`) . . . 12
4.3 CS-WOTS$^+$ Private Key (Function `wots_SKgen`) . . . 13
4.4 CS-WOTS$^+$ Public Key Generation (Function `wots_PKgen`) . . . 13
4.5 CS-WOTS$^+$ Encoding (Function `wots_encode`) . . . 15
4.6 CS-WOTS$^+$ Signature Generation (Function `wots_sign`) . . . 15
4.7 CS-WOTS$^+$ Compute Public Key from Signature (Function `wots_pkFromSig`) . . . 16

5 The Kylin Hypertree **16**
5.1 (Fixed Input-Length) XMSS . . . 17
5.1.1 XMSS Parameters . . . 17
5.1.2 XMSS Private Key . . . 17
5.1.3 TreeHash (Function `treehash`) . . . 17
5.1.4 XMSS Public Key Generation (Function `xmss_PKgen`) . . . 18
5.1.5 XMSS Signature . . . 18
5.1.6 XMSS Signature Generation (Function `xmss_sign`) . . . 19
5.1.7 XMSS Compute Public Key from Signature (Function `xmss_pkFromSig`) . . . 19
5.2 HT: The Hypertree . . . 19
5.2.1 HT Parameters . . . 21
5.2.2 HT Key Generation (Function `ht_PKgen`) . . . 21
5.2.3 HT Signature . . . 21
5.2.4 HT Signature Generation (Function `ht_sign`) . . . 21
5.2.5 HT Signature Verification (Function `ht_verify`) . . . 22

6 FORS: Forest Of Random Subsets **22**
6.1 FORS Parameters . . . 24
6.2 FORS Private Key (Function `fors_SKgen`) . . . 24
6.3 FORS TreeHash (Function `fors_treehash`) . . . 25
6.4 FORS Public Key (Function `fors_PKgen`) . . . 26
6.5 FORS Signature Generation (Function `fors_sign`) . . . 26
6.6 FORS Compute Public Key from Signature (Function `fors_pkFromSig`) . . . 26

7 The Full Kylin Signature Scheme **27**
7.1 Kylin Parameters . . . 27
7.2 Kylin Key Generation (Function `kylin_keygen`) . . . 29
7.3 Kylin Signature . . . 29
7.4 Kylin Signature Generation (Function `kylin_sign`) . . . 29
7.5 Kylin Signature Verification (Function `kylin_verify`) . . . 30

8 Parameter Sets and Hash/XOF Instantiation **31**
8.1 Kylin Parameter Sets . . . 31
8.1.1 Influence of Parameters on Security and Performance . . . 32
8.1.2 Proposed Parameter Sets and Security Levels . . . 33
8.2 Instantiations of Hash Functions . . . 33

9 Design Rationale **34**

10 Security Statements and Analyses **35**

11 Performance Evaluation **36**

12 Feature Statements **37**

A Parameter Evaluation Sage Script **39**

B $q = 2^{24}$ Companion Parameter Profiles **42**

1 Algorithm Description

Hash-based signature is one of the most promising candidates for (and perhaps the most conservative approach to) post-quantum digital signatures. An advantage of hash-based signatures is that its (classical as well as quantum) security strength is better understood (and easier to evaluate) than other candidates, by solely relying on the idealized hardness[1] of the cryptographic hash functions.

Stateful signatures. Ralph Merkle proposed a hash-based signature [21] that builds upon Lamport's one-time signature (OTS) [19]. The recent efforts towards improving stateful signatures lead to the eXtended Merkle Signature Scheme (XMSS) [15] and the Leighton-Micali Signature (LMS) [20], standardized by NIST [8] and IETF.

Stateless signatures. In a typical stateful signature, e.g., Merkle's signature scheme (MSS), the signer keeps track of which private key of the OTS has been used to avoid security issues from subsequent reuse. This is however not always possible in many practical scenarios. Goldreich proposed a stateless hash-based signature construction [11, 12] which removes the need for maintaining a local state but results in prohibitively large signatures. Recently, this line of research has renewed interest. By incorporating the hypertree structure, SPHINCS [4] offered a practical instantiation of the Goldreich-style stateless hash-based signature. SPHINCS serves as a basis for subsequent works driven by the NIST PQC standardization process, including Gravity-SPHINCS [2], SPHINCS-Simpira [13], and SPHINCS$^+$ [5]. More recent optimizations and analyses include SPHINCS+C [16], improved size-time tradeoffs in hypertree-based settings [17], variable-height/chain-length framework extensions [25], and the SPHINCS-alpha work revisiting constant-sum Winternitz one-time signatures with applications to SPHINCS+ and XMSS [30]. NIST has also published the finalized federal standard for stateless hash-based signatures (SLH-DSA) in FIPS 205 [22].

Table 1: Hash-call and size metrics for the three packaged Kylin families. Key generation, signing, and verification are reported in hash-call proxies; key and signature sizes are in bytes.

Instance	KeyGen	Sign	Verify	Pk	Sk	Sig
Kylin-128	417279	2442880	7357	32	64	6688
Kylin-256	324095	3223168	12577	64	128	26784
Kylin-512	495615	6198272	42476	128	256	103808

Summary of our Work. We propose Kylin, a stateless hash-based signature scheme, which improves upon the state-of-the-art stateless hash-based signature scheme SPHINCS$^+$, while preserving the core elements that made the original SPHINCS$^+$ a standout project.

Our optimization mainly stems from the optimization of the one-time signature scheme, which we prove to have a size-optimal encoding scheme among all tree structured one-time signatures. Moreover, no DAG-based construction offers better size efficiency, making our proposal size-optimal among all known schemes to the best of our knowledge [30].

Since our modification upon the SPHINCS$^+$ scheme is limited to the encoding scheme of the underlying WOTS$^+$one-time signature, the security analysis of our new scheme remains largely the same as the original analysis. Based on the security analysis of the original work, we estimate the security level and carefully choose parameters in order to achieve optimized performance.

To facilitate a direct package-level view, Table 1 reports the hash-call proxies and key/signature sizes for `Kylin-128`, `Kylin-256`, and `Kylin-512`. In particular, the table includes key-generation, signing, and verification hash-call metrics under the frozen packaged parameter sets.

We refer to more detailed comparisons for the full spectrum of parameter choices in Section 11. As summarized in Table 5, the packaged Kylin instances provide the current repository-local performance profile for key generation, signing, and verification. For specific scenarios where verification time is critical, the parameters can be re-tuned towards faster verification.

[1] The design philosophy of symmetric primitives (including hash functions) is that they should only admit generic attacks, otherwise the design is considered to be flawed.

Acknowledgement. We acknowledge the SPHINCS$^+$ team for their great work on the original SPHINCS$^+$ project. A significant portion of our code and documentation derives from their commendable efforts.

2 Main Design Differences from SPHINCS$^+$

In this section we highlight our improvement upon the original SPHINCS$^+$ scheme, namely, the one-time signature component. Our optimized scheme, CS-WOTS$^+$, allows a larger message space / signature size ratio as compared to the original WOTS$^+$ scheme, allowing a smaller one-time signature size. As a result, the parameters of the hash-based signature scheme can thus be re-tuned to reduce the signature size without exacerbating the computation time. In addition, Kylin uses a more flexible hypertree layout with diverse per-layer heights, which lets the packaged parameter sets remain well balanced even when h is not divisible by d.

We explain the details of our optimization as well as some technical proofs in this section.

2.1 Diverse Layer Heights in the Hypertree

Earlier parameter profiles often imposed the divisibility condition $h \bmod d = 0$ so that every layer had the same height h/d. In this draft, we allow per-layer heights $(h_0, \ldots, h_{d-1})$ to be non-uniform as long as $\sum_{i=0}^{d-1} h_i = h$. By default, we assume the layer-height vector is ordered, i.e., $h_0 \le h_1 \le \cdots \le h_{d-1}$.

Let $r = h \bmod d$. Our default balancing rule is:

- choose $d - r$ layers with height $\lfloor h/d \rfloor$,
- choose r layers with height $\lceil h/d \rceil$.

Equivalently, the profile is as balanced as possible and differs by at most one between any two layer heights.

2.2 The WOTS$^+$ Encoding

The reason that the WOTS$^+$(as well as other Winternitz-type OTS) scheme introduces the checksum is that in absence of the checksum the adversary can efficiently forge signatures given a single pair of valid message signature. That is, given (σ, m) he forges any m' satisfying $\forall i(m_i \le m'_i)$ by computing $\mathbf{F}^{m'_i}(sk_i) = \mathbf{F}^{m'_i - m_i}\big(\mathbf{F}^{m_i}(sk_i)\big)$.

The checksum addresses the issue: an increase in any m_i leads to a decrease in at least one C_i (recall $C = \sum_{i=1}^{l_1}(w - 1 - m_i)$). Therefore, the adversary cannot forge any (m', C') satisfying both $\forall i(m_i \le m'_i)$ and $\forall i(C_i \le C'_i)$ at the same time.

2.3 Size-Optimal Encoding

More formally, the problem of constructing one-time signature reduces to that of building an efficient encoding scheme $\mathsf{Enc} : \mathcal{M} \to \mathcal{C} \subseteq [w]^l$ for some incomparable codeword set $\mathcal{C}$ (see Definition 1). In case of WOTS$^+$, the encoding function Enc simply appends the checksum to the original message. Note that WOTS$^+$ fixes the size of the message to l_1 (i.e., $\mathcal{M} = [w]^{l_1}$) and then constructs as small codewords as possible (minimizing $l - l_1$).

Definition 1 ((In)comparability)**.** *For $a, b \in [w]^l$, we denote by $a \le b$ if for every $i \in [l]$ we have $a_{i+1} \le b_{i+1}$. If $a \le b$ or $b \le a$ we say that a and b comparable, or otherwise a and b are incomparable. A set $S \subseteq \{a : a \in [w]^l\}$ is said to be incomparable (or called an "antichain" in order theory terminology) if any two elements of S are incomparable.*

We take a slightly different approach to encoding the messages. That is, we first fix the size of the codewords to l, $\mathcal{C} \subseteq [w]^l$, and strive to accommodate as large message space $\mathcal{M}$ as possible. Given that Enc is an injection it is essentially to maximize the size of $\mathcal{C} \subseteq [w]^l$. A natural approach is to encode the codewords such that all elements of every codeword sum to the same value, and therefore the checksum is not explicitly needed. Bos and Chaum [7] studied this approach for the special case of $w = 2$. Vaudenay [28] generalized it to arbitrary w, but he did not provide an encoding algorithm. Perin et al. provided a similar encoding algorithm in [24], but they did not present a size-optimal proof.

Theorem 2.1. *For any $m \in [l(w-1)+1]$, $\mathcal{C}_m \stackrel{\text{def}}{=} \{v \in [w]^l : \sum_{i=1}^{l} v_i = m\}$ is incomparable.*

Proof. Suppose towards contradiction that $\mathcal{C}_m$ (for some fixed $m \in [l(w-1)+1]$) is not incomparable, then there exist distinct $a, b \in \mathcal{C}_m$ s.t. $a \leq b$. There must be an index j such that $a_j < b_j$ (otherwise $a = b$). However, due to equal sum $\sum_i a_i = \sum_i b_i$ we have $\sum_{1 \leq i \leq l \wedge i \neq j}(a_i - b_i) > 0$, and there must exist some $1 \leq k \leq l$ such that $a_k > b_k$, which is a contradiction to $a \leq b$. □ □

Every $\mathcal{C}_m$ gives an encoding scheme but with different size. For $m = 0$ or $m = l(w-1)$, $\mathcal{C}_m$ consists of only a single codeword. We argue that the size of $\mathcal{C}_m$ reaches its maximal in the middle, i.e., when $m = \lfloor \frac{l(w-1)}{2} \rfloor$. One easily verifies that this holds in the binary case (i.e., $w = 2$) where $|\mathcal{C}_m| = \binom{l}{m}$. Perin et al. [24] proved that $|\mathcal{C}_m|$ reaches its maximum when $m = \lfloor \frac{l(w-1)}{2} \rfloor$. We prove a stronger optimality result in Theorem 2.2 that the size of $\mathcal{C}_m$, when $m = \lfloor \frac{l(w-1)}{2} \rfloor$, is not only the largest in all $\mathcal{C}_m$ for $m \in [l(w-1)+1]$ but the largest among all valid sets of codewords.

Theorem 2.2 (Size-optimal encoding)**.** *For every incomparable $\mathcal{C}^* \in P([w]^l)$, it holds that*

$$|\mathcal{C}^*| \leq |\mathcal{C}_{\lfloor \frac{l(w-1)}{2} \rfloor}| \ .$$

We defer its proof to Theorem 2.4, which rephrases Theorem 2.2 in the language of order theory. Prior to that, we discuss how to compute $|\mathcal{C}_m|$ by recursion, and give an explicit construction of encoding messages into $\mathcal{C}_m$ for $m = \lfloor \frac{l(w-1)}{2} \rfloor$. Hereafter, we denote such $\mathcal{C}_m$ with maximal size by $\mathcal{C}$ for brevity.

Counting the size. Now we need to figure out the size of $\mathcal{C}$. As a special case, $|\mathcal{C}| = \binom{l}{\lfloor l/2 \rfloor}$ when $w = 2$. Fix w, let

$$D_{n,m} = |\{v \in [w]^n : \sum_{i=1}^{n} v_i = m\}| \ ,$$

we have their initial values

$$D_{1,m} = 1, \text{ for } m \in \{0, 1, \ldots, w-1\}$$
$$D_{n,m} = 0, \text{ for } 2 \leq n \in \mathbb{Z}, m \in \mathbb{Z}^- \ ,$$

and recurrence relation

$$D_{n,m} = \sum_{i=0}^{w-1} D_{n-1,m-i}, 2 \leq n \in \mathbb{Z}, m \in \{0, 1, \ldots, l(w-1)\} \ . \tag{1}$$

Note when $w = 2$, this method is equivalent to recurrence relation of binomial coefficient, i.e., $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$.

Let us explain the recurrence relation. To compute $D_{n,m}$, consider the value of its last summand, which could be any value in $\{0, 1, \ldots, w-1\}$. If this value is set to i, the sum of the first $n-1$ elements must be $m - i$. Therefore, we notice that the problem "n elements with sum to m" into those "$n-1$ elements with sum to $m - i$". Thus we can simply count $D_{n,m}$ by accumulating $D_{n-1,m-i}$. Following this method, $D_{l,\lfloor l(w-1)/2 \rfloor}$ gives the size of $\mathcal{C}$.

Related works. $D_{n,m}$ is also the m-th coefficient of $(1 + x + x^2 + \cdots + x^{w-1})^n$. Euler [10] has studied $w = 3, 4, 5$, known as trinomial, quadrinomial and quintinomial coefficients respectively. The generalized form was studied in the literature, e.g., [1, 29, 3]. Actually, we can use an inclusion-exclusion argument to express it as a function of binomial coefficients [14]

$$D_{n,m} = \sum_{s=0}^{\lfloor m/w \rfloor} (-1)^s \binom{n}{s} \binom{m + n - sw - 1}{n - 1} \ .$$

Encoding algorithm. Now we make the construction explicit by giving an efficient encoding algorithm, which maps a message $x \in [|\mathcal{C}|]$ into an element in $\mathcal{C}$. We give the pseudocode of the encoding algorithm in Algorithm 1.

Let us explain the encoding algorithm. As previously stated, the problem can be divided into several subproblems by considering the value of the first element v_{l-i}. To encode a natural number $x \in [0, D_{i,m})$, we can simply determine $v_{l-i} = j$ by seeking which j satisfies $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k \leq j} D_{i-1,m-k})$. Once the value of v_{l-i} is determined, we proceed to the next terms until all elements are decided.

In order to prove the optimality of the encoding, we need some prerequisites about the order theory. The relationship between one-time signature and order theory has been investigated in [6].

Algorithm 1: Encode:$[|\mathcal{C}|] \to \mathcal{C}$.

```
Function Encode(x)
    Let v be an array of size l;
    m ← ⌊l(w − 1)/2⌋;
    for i ← l ... 1 do
        for j ← 0 ... min(w − 1, m) do
            if x ≥ D_{i−1,s−j} then
                x ← x − D_{i−1,s−j};
            else
                v_{l−i} ← j;
                break;
        m ← m − v_{l−i};
    return v;
```

Preliminaries of Order Theory

Definition 2 (Poset)**.** *A poset ($\mathcal{S}$,$\leq$) consists of a set $\mathcal{S}$ together with an antisymmetric, transitive and reflexive binary relation '$\leq$', where are certain pairs $(x, y) \in \mathcal{S}$ are comparable ($x \leq y$ or $y \leq x$).*

Note that a poset does not require all pairs in $\mathcal{S}$ to be comparable, and thus it is also known as a partially ordered set.

Definition 3 ((Anti)chain and decomposition)**.** *A chain (resp., antichain) refers to a subset of a poset, whose every pair of elements is comparable (resp., incomparable). A chain decomposition is a partition of a poset into disjoint chains.*

Theorem 2.3 (Dilworth's theorem [9])**.** *For any finite poset P, the size of P's maximum antichain equals the size of a minimum chain decomposition of P.*

Let $P = ([w]^l, \leq)$ be a finite poset. According to Dilworth's theorem, we can prove that $\mathcal{C}$ is the maximum antichain of P by arguing that (1) $\mathcal{C}$ is an antichain and (2) we can find a chain decomposition whose size equals to $|\mathcal{C}|$.

Theorem 2.4. *$\mathcal{C}$ is the maximum antichain, i.e., $\mathcal{C}$ is size-optimal.*

Proof. We have proved that $\mathcal{C}$ is an antichain in Theorem 2.1. It remains to construct the chain decomposition of size $|\mathcal{C}|$ as follows. Our proof can be viewed as a generalization of the proof of Sperner's theorem [27], which considers the special case for $w = 2$.

Consider poset $S_n = ([w]^n, \leq)$, and we sometimes denote an element of S_n by $(a_1, ..., a_n) \in [w]^n$ or by $c_i \in [w]^n$. We slightly abuse the notation $|(a_1, ..., a_n)| \stackrel{\text{def}}{=} a_1 + ... + a_n$.

We construct the chain decomposition for S_n by induction, where every chain $\{c_1, \ldots, c_m\}$ satisfies the following two properties:

- $|c_{i+1}| = |c_i| + 1, \forall i \in \{1, 2, \ldots, m-1\}$,
- $|c_1| + |c_m| = n \cdot (w-1)$.

The case for $n = 1$ is trivial, i.e., $D_{1,\lfloor (w-1)/2 \rfloor} = 1$, which correspond to the chain of $S_1 = \{(0), (1), \ldots, (w-1)\}$.

Assume that we have a chain decomposition for S_{n-1} satisfying the above two properties, we proceed to the construction of a chain decomposition for S_n. For each chain $c = \{c_1, c_2, \ldots, c_m\}$ (from the chain decomposition of S_{n-1}) satisfying the two properties, we build $k+1$ chains for S_n as follows, where $k = \min(w-1, m-1)$. That is, for every $j \in \{0, ..., k\}$ the j-th chain consists of:

$$(c_1, j) \leq \ldots \leq (c_{m-j}, j) \leq (c_{m-j}, j+1) \leq ... \leq (c_{m-j}, w-1) \ .$$

This yields the $k+1$ chains as shown in Fig. 1 below:

It is easy to verify that $|(c_1, j)| + |(c_{m-j}, w-1)| = |(c_1, 0)| + j + |(c_m, 0)| - j + (w-1) = n(w-1)$, and every subsequent element increase the sum value of its predecessor by one. Namely, the two properties are preserved for all the constructed chains of S_n.

$$
\begin{array}{ccccccc}
(c_1,0) & \dots & \dots & \dots & (c_m,0) & \dots & (c_m,w-1) \\
\vdots & \dots & \dots & \iddots & \dots & \dots & \vdots \\
(c_1,k) & \dots & (c_{m-k},k) & \dots & \dots & \dots & (c_{m-k},w-1)
\end{array}
$$

Figure 1: A demonstration of how a chain from S_{n-1} is expanded into $k+1$ chains for S_n, where every row is an expanded chain. Note that it is not a rectangular matrix (every row has two less elements than the previous).

Table 2: Comparison of length l between WOTS$^+$ and CS-WOTS$^+$ for different values of Winternitz parameters w and security parameter λ.

	128bit		192bit		256bit	
w	WOTS$^+$	Ours	WOTS$^+$	Ours	WOTS$^+$	Ours
8	46	45	67	66	90	88
16	35	34	51	50	67	66
24	31	30	45	44	59	58
32	28	27	42	40	55	53
40	27	26	39	38	52	50
48	25	25	37	36	48	48

It remains to argue that all the chains constructed (from the decomposed chains of S_{n-1}) constitute a partition of $[w]^n$. That is, for every $c_i \in S_{n-1}$, each of its augmented elements $(c_i,0),...,(c_i,w-1)$ appears in the constructed chains exactly once. Note that every c_i belongs to (and can only belong to) one of the decomposed chains of S_{n-1}, say $c=\{c_1,\dots,c_m\}$. We discuss the following cases.

Case $m \le w$: We have $k=m-1 \le w-1$. $[(c_i,0),\dots,(c_i,k+1-i)]$ appears as the first $(k+2-i)$ elements of the i-th column, and then $[(c_i,k+1-i),\dots,(c_i,w-1)]^T$ as the last $(w+i-k-1)$ elements of the $(k+2-i)$-th row in Fig. 1.

Case $m > w$: We have $k=w-1<m-1$. If $1 \le i \le m-w+1$, then $[(c_i,0),(c_i,w-1)]$ appears as the i-th column in Fig. 1. Otherwise, $m-w+1 < i \le m$. $[(c_i,0),\dots,(c_i,m-i)]$ and $[(c_i,m-i),\dots,(c_i,w-1)]^T$ are the first $m-i+1$ elements of the i-th column, and the last $(w+i-m)$ elements of the $(m-i+1)$-th row respectively.

Therefore, we have shown that for every $a \in [w]^{n-1}$, $(a,0)$, ...,$(a,w-1)$ appears exactly once in the newly constructed chains, namely, the chains constitutes as a chain decomposition for S_n. Finally, it remains to count the number of chains in the decomposition. The two properties guarantee that every chain contains exactly one element c_{mid} with $|c_{\mathsf{mid}}| = \lfloor l(w-1)/2 \rfloor$ (i.e., $c_{\mathsf{mid}} \in \mathcal{C}$). Thus, the size of chain decomposition $|\mathcal{C}| = D_{l,\lfloor l(w-1)/2 \rfloor}$. This completes the proof that $\mathcal{C}$ is the maximum antichain.

□

2.4 Theoretical Performance

Constant Sum WOTS$^+$ has two advantages over WOTS$^+$.

- Stable computing time. The number of hash function calls is fixed in our construction, in contrast to possibly variable numbers for the signing and verification algorithm of WOTS$^+$. While no timing attacks are identified against the implementations of our construction and WOTS$^+$, stable computing time is always preferable (especially for signing algorithms whose computation involves a private key).
- Reduced signature size and number of hash calls. For instance, a baseline 256-bit-classical parameter choice suggests $w=16$ and $l=67$. In our construction, for $w=16$ we require $l=66$, which reduces 1.5% in both running time (in terms of the expected number of hash function calls) and size. We refer to Table 2 for more details.

Although our encoding algorithm costs slightly more than the checksum method, it is less dominant compared to the number of hash function calls used in the signature scheme, which will be confirmed in the performance comparison to SPHINCS$^+$.

3 Notation and Supporting Definitions

In the following we start defining basic mathematical operations on integers and bit strings. From that we work our way to more specific basic methods used later in the specification.

3.1 Data Types

Bytes and byte strings are the fundamental data types. A byte is a sequence of eight bits. The set of bytes is denoted as $\mathbb{B}$. A single byte is denoted as a pair of hexadecimal digits with a leading "`0x`". A byte string is an ordered sequence of zero or more bytes and is denoted as an ordered sequence of hexadecimal characters with a leading "`0x`". For example, `0xe534f0` is a byte string of length 3. An array of byte strings is an ordered, indexed set starting with index 0 in which all byte strings have identical length. We assume big-endian representation for any data types or structures.

3.2 Functions

We define the following functions:

$\lceil x \rceil$ (or $\mathsf{ceil}(x)$) for x a real number returns the smallest integer greater than or equal to x.

$\lfloor x \rfloor$ (or $\mathsf{floor}(x)$) for x a real number returns the largest integer less than or equal to x.

$\log(x)$ for x a non-negative real number returns the logarithm to base 2 of x. In pseudocode this function is written as lg.

$\mathsf{Trunc}_\ell(x)$ truncates the bit-string x to the first ℓ bits.

3.3 Operations

When a and b are integers, mathematical operators are defined as follows:

- $\hat{}$: a^b denotes the result of a raised to the power of b.
- $\cdot$: $a \cdot b$ denotes the product of a and b. This operator is sometimes omitted in the absence of ambiguity, as in usual mathematical notation.
- $/$: a/b denotes the quotient of a by non-zero b.
- $\%$: $a\%b$ denotes the non-negative remainder of the integer division of a by b.
- $+$: $a + b$ denotes the sum of a and b.
- $-$: $a - b$ denotes the difference of a and b.
- $++$: $a++$ denotes incrementing a by 1, i.e., $a = a + 1$.
- $<<$: $a << b$ denotes a logical left shift of a by b positions, for b being non-negative, i.e., $a \cdot 2^b$.
- $>>$: $a >> b$ denotes a logical right shift of a by b positions, for b being non-negative, i.e., $\mathsf{floor}(a/2^b)$.

The standard order of operations is used when evaluating arithmetic expressions.

Arrays are used in the common way, where the i-th element of an array A is denoted $A[i]$. Byte strings are treated as arrays of bytes where necessary: If X is a byte string, then $X[i]$ denotes its i-th byte, where $X[0]$ is the leftmost, highest order byte.

If A and B are byte strings of equal length, then:

- A AND B denotes the bitwise logical conjunction operation.
- A XOR B (or $A \oplus B$) denotes the bitwise logical exclusive disjunction operation.

When B is a byte and i is an integer, then $B >> i$ denotes the logical right-shift by i positions.

If X is an x-byte string and Y a y-byte string, then $X \| Y$ denotes the concatenation of X and Y,with $X \| Y = X[0]...X[x-1]Y[0]...Y[y-1]$.

3.4 Integer to Byte Conversion (Function `toByte`)

For x and y non-negative integers, we define $Z = \texttt{toByte}(x, y)$ to be the y-byte string containing the binary representation of x in big-endian byte-order.

3.5 Member Functions (Functions `set, get`)

To simplify algorithm descriptions, we assume the existence of member functions. If a complex data structure like a public key `PK` contains a variable `X` then `PK.getX()` returns the value of `X` for this public key. Accordingly, `PK.setX(Y)` sets variable `X` in `PK` to the value held by `Y`. Since camelCase is used for member function names, a value `z` may be referred to as `Z` in the function name, e.g., `getZ`.

3.6 Cryptographic (Hash) Function Families

3.6.1 Tweakable Hash Functions (Functions `T, F, H`)

Following SPHINCS$^+$, a tweakable hash function takes a public seed `PK.seed` and an address `ADRS` in addition to the message input. This effectively renders hash function calls for each key pair and position in the virtual tree independent. The addressing scheme will be described in Section 3.6.3.

The schemes described in this specification build upon several instantiations of tweakable hash functions of the form

$$\begin{aligned} \mathbf{T}_\ell &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{\ell n} \rightarrow \mathbb{B}^n \\ \texttt{md} &\leftarrow \mathbf{T}_\ell(\texttt{PK.seed}, \texttt{ADRS}, M) \ , \end{aligned}$$

mapping and ℓn-byte message M to an n-byte hash value md using an n-byte seed `PK.seed` and a 32-byte address `ADRS`. The function $\mathbf{T}_\ell$ is denoted by `T_l` in pseudocode.

There are two special cases which we rename for consistency with previous descriptions of hash-based signature schemes:

$$\begin{aligned} \mathbf{F} &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n \\ \mathbf{F} &\stackrel{\text{def}}{=} \mathbf{T}_1 \\ \mathbf{H} &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \rightarrow \mathbb{B}^n \\ \mathbf{H} &\stackrel{\text{def}}{=} \mathbf{T}_2 \end{aligned}$$

3.6.2 PRF and Message Digest (Functions `PRF, PRF_msg, H_msg`)

Kylin uses a pseudorandom function **PRF** for pseudorandom key generation:

$$\mathbf{PRF} : \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n \ .$$

In addition, Kylin uses a pseudorandom function $\mathbf{PRF_{msg}}$ to generate randomness for the message compression:

$$\mathbf{PRF_{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n \ .$$

To compress the message to be signed, Kylin uses an additional keyed hash function $\mathbf{H_{msg}}$ that can process arbitrary length messages:

$$\mathbf{H_{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^m \ .$$

3.6.3 Hash Function Address Scheme (Structure of `ADRS`)

An address `ADRS` is a 32-byte value that follows a defined structure. In addition, it comes with set methods to manipulate the address. We explain the generation of addresses in the following sections where they are used. Essentially, all functions have to keep track of the current context, updating the addresses after each hash call.

There are five different types of addresses for the different use cases. One type is used for the hashes in CS-WOTS$^+$ schemes, one is used for compression of the CS-WOTS$^+$ public key, the third is used for hashes within the main Merkle tree construction, another is used for the hashes in the Merkle tree in

FORS, and the last is used for the compression of the tree roots of FORS. These types largely share a common format. We describe them in more detail, below.

The structure of an address complies with word borders, with a word being 32 bits long in this context. Only the tree address (i.e. the index of a specific subtree in the main tree) is too long to fit a single word: for this, we reserve three words. An address is structured as follows. It always starts with a layer address of one word in the most significant bits, followed by a tree address of three words. These addresses describe the position of a tree within the hypertree. The layer address describes the height of a tree within the hypertree starting from height zero for trees on the bottom layer. The tree address describes the position of a tree within a layer of a multi-tree starting with index zero for the leftmost tree. The next word defines the type of the address. It is set to 0 for a CS-WOTS$^+$ hash address, to 1 for the compression of the CS-WOTS$^+$ public key, to 2 for a hash tree address, to 3 for a FORS address, and to 4 for the compression of FORS tree roots.

We first describe the CS-WOTS$^+$ address (Fig. 2). In this case, the type word is followed by the key pair address that encodes the index of the CS-WOTS$^+$ key pair within the specified tree. The next word encodes the chain address (i.e. the index of the chain within CS-WOTS$^+$), followed by a word that encodes the address of the hash function call within the chain. Note that for the generation of the secret keys based on `SK.seed` a different type of address is used (see below).

Layer Address	Tree Address		
type = 0	Key Pair Address	Chain Address	Hash Address

Figure 2: CS-WOTS$^+$ hash address.

The second type (Fig. 3) is used to compress the CS-WOTS$^+$ public keys. The type word is set to 1. Similar to the address used within CS-WOTS$^+$, the next word encodes the key pair address. The remaining two words are not needed, and thus remain zero. We zero pad the address to the constant length of 32 bytes.

Layer Address	Tree Address	
type = 1	Key Pair Address	Padding = 0

Figure 3: CS-WOTS$^+$ public key compression address.

The third type (Fig. 4) addresses the hash functions in the main tree. In this case the type word is set to 2, followed by a zero padding of one word. The next word encodes the height of the tree node that is being computed, followed by a word that encodes the index of this node at that height.

Layer Address	Tree Address		
type = 2	Padding = 0	Tree Height	Tree Index

Figure 4: hash tree address.

The next type (Fig. 5) is of a similar format, and is used to describe the hash functions in the FORS tree. The type word is set to 3. The key pair address is used to signify which FORS key pair is used, identical to the key pair address in the CS-WOTS$^+$ hash addresses. Its value is the same as that of the CS-WOTS$^+$ key pair that is used to authenticate it, i.e. its index as a leaf in the specified tree. The tree height and tree index fields are used to address the hashes within the FORS tree. This is done like for the above-mentioned hashes in the main tree, with the additional consideration that the tree indices are counted continuously across the different FORS trees. To generate the leaf nodes from `SK.seed` a different type of address is used (see below).

The next type (Fig. 6) is used to compress the tree roots of the FORS trees. The type word is set to 4. Like the CS-WOTS$^+$ public key compression address, it contains only the address of the FORS key pair, but is padded to the full length.

The final two types are used for secret key value generation in CS-WOTS$^+$ and FORS. A CS-WOTS$^+$ key generation address (Fig. 7) is the same as a CS-WOTS$^+$ hash address with two differences. First, the

Layer Address	Tree Address		
type = 3	Key Pair Address	Tree Height	Tree Index

Figure 5: FORS tree address.

Layer Address	Tree Address	
type = 4	Key Pair Address	Padding = 0

Figure 6: FORS tree roots compression address.

type word is set to 5. Second, the hash address word is constantly set to 0. When generating the secret key value for a given chain, the remaining words have to be set the same way as for the CS-WOTS$^+$ hash addresses used for this chain.

Layer Address	Tree Address		
type = 5	Key Pair Address	Chain Address	Hash Address = 0

Figure 7: CS-WOTS$^+$ key generation address.

Similarly, the FORS key generation type (Fig. 8) is the same as the FORS tree address type, except that the type word is set to 6, and the tree height word is set to 0. As for the CS-WOTS$^+$ key generation address, the remaining words have to be set as for the FORS tree address used when processing the generated value.

All fields within these addresses encode unsigned integers. When describing the generation of addresses we use set methods that take positive integers and set the bits of a field to the binary representation of that integer, in big-endian notation. Throughout this document, we adhere to the convention of assuming that changing the type word of an address (indicated by the use of the `setType()` method) initializes the subsequent three words to zero.

In order to make keeping track of the types easier throughout the pseudo-code in the rest of this document, we refer to them respectively using the constants `WOTS_HASH`, `WOTS_PK`, `TREE`, `FORS_TREE`, `FORS_ROOTS`, `WOTS_PRF`, and `FORS_PRF`.

4 CS-WOTS$^+$ One-Time Signatures

This section describes the CS-WOTS$^+$ scheme, while a private key can be used to sign any message, each private key MUST NOT be used to sign more than a single message.

4.1 CS-WOTS$^+$ Parameters

CS-WOTS$^+$ uses the parameters n, w and `len`; They are positive integer values. These parameters are summarized as follows:

- n: the security parameter; it is the message length as well as the length of a private key, public key, or signature element in bytes.
- w: the Winternitz parameter;
- `len`: the number of n-byte-string elements in a CS-WOTS$^+$ private key, public key, and signature.

4.2 CS-WOTS$^+$ Chaining Function (Function `chain`)

The chaining function (Algorithm 2) computes an iteration of **F** on an n-byte input using a CS-WOTS$^+$ hash address `ADRS` and a public seed `PK.seed`. The address `ADRS` MUST have the first seven 32-bit words set to encode the address of this chain. In each iteration, the address is updated to encode the current position in the chain before `ADRS` is used to process the input by **F**.

Layer Address	Tree Address		
type = 6	Key Pair Address	Tree Height = 0	Tree Index

Figure 8: FORS key generation address.

In the following, ADRS is a 32-byte CS-WOTS$^+$ hash address as specified in Section 3.6.3 and PK.seed is a n-byte string. The chaining function takes as input an n-byte string X, a start index i, a number of steps s, as well as ADRS and PK.seed. The chaining function returns as output the value obtained by iterating **F** for s times on input X.

Algorithm 2: chain — Chaining function used in CS-WOTS$^+$.

```
# Input: Input string X, start index i, number of steps s, public seed PK.seed,
  ↪ address ADRS
# Output: value of F iterated s times on X
chain(X, i, s, PK.seed, ADRS) {
  if ( s == 0 ) {
    return X;
  }
  if ( (i + s) > (w - 1) ) {
    return NULL;
  }
  byte[n] tmp = chain(X, i, s - 1, PK.seed, ADRS);
  ADRS.setHashAddress(i + s - 1);
  tmp = F(PK.seed, ADRS, tmp);
  return tmp;
}
```

4.3 CS-WOTS$^+$ Private Key (Function wots_SKgen)

The CS-WOTS$^+$ private key, denoted by SK, is a length len array of n-byte strings. This private key MUST NOT be used to sign more than one message. This private key is only implicitly used. Therefore, the following is just to support a better understanding of the following algorithms. Each n-byte string in the CS-WOTS$^+$ private key is derived from a secret seed SK.seed which is part of the Kylin secret key and a CS-WOTS$^+$ key generation address skADRS using **PRF**. The same secret seed is used to generate all secret key values within Kylin. The address used to generate the i-th n-byte string of SK MUST encode the position of the i-th hash chain of this CS-WOTS$^+$ instance within the Kylin structure.

The following pseudocode (Algorithm 3) describes an algorithm to generate a CS-WOTS$^+$ private key.

4.4 CS-WOTS$^+$ Public Key Generation (Function wots_PKgen)

A CS-WOTS$^+$ key pair defines a virtual structure that consists of len hash chains of length w. Each of the len stings of n-bytes in the private key defines the start node for one hash chain. The public key is the tweakable hash of the end nodes of these hash chains. To compute the hash chains, the chaining function (Algorithm 2) is used. A CS-WOTS$^+$ hash address ADRS and a seed PK.seed have to be provided by the calling algorithm as well as a secret seed SK.seed. The address ADRS MUST encode the address of the CS-WOTS$^+$ key pair within the Kylin structure. Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of ADRS other than the last three 32-bit words. Note that the PK.seed used here is public information also available to a verifier.

The following pseudocode (Algorithm 4) describes an algorithm for generating the public key PK.

Algorithm 3: `wots_SKgen` — Generating a CS-WOTS$^+$ private key.

```
# Input: secret seed SK.seed, address ADRS
# Output: CS-WOTS+ private key sk
wots_SKgen(SK.seed, ADRS) {
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
  }
  return sk;
}
```

Algorithm 4: `wots_PKgen` — Generating a CS-WOTS$^+$ public key.

```
# Input: secret seed SK.seed, address ADRS, public seed PK.seed
# Output: CS-WOTS+ public key pk
wots_PKgen(SK.seed, PK.seed, ADRS) {
  wotspkADRS = ADRS; // copy address to create OTS public key address
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
  skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    tmp[i] = chain(sk[i], 0, w - 1, PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_len(PK.seed, wotspkADRS, tmp);
  return pk;
}
```

4.5 CS-WOTS$^+$ Encoding (Function `wots_encode`)

Let us explain the encoding algorithm (Algorithm 5). The problem can be divided into several subproblems by considering the value of the first element c_{l-i}. To encode a natural number $x \in [0, D_{i,m})$, we can simply determine $c_{l-i} = j$ by seeking which j satisfies $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k\leq j} D_{i-1,m-k})$. Once the value of v_{l-i} is determined, we proceed to its next terms until all elements are decided. All arithmetic operations MUST be compute in unsigned 256-bit. Array D can be precomputed to avoid computing D for each encoding function call.

Algorithm 5: `wots_encode` — Encoding message to constant-sum encoding.

```
# Input: Message M
# Output: Constant-Sum Encoding of M
wots_encode(M) {

  s=floor(len*(w-1)/2)
  Let D be an 2d array of size (len+1)*(s+1) initialized to 0

  D[0][0]=1
  for( i = 1 ; i <= len ; i++ ){
    for( j = 0 j <= s ; j++ ){
      for( k = 0 ; k < w and k <= j ; k++ ){
        D[i][j] = D[i][j] + D[i-1][j-k]
      }
    }
  }
  // array D can be precomputed

  Let c be an array of size len
  for( i = len ; i >= 1 ; i-- ){
    for( j = 0 ; j < w and j <= s  ; j++ ){
      if( M >= D[i-1][s-j] ){
        M = M - D[i-1][s-j]
      } else {
        c[len-i] = j
        break
      }
    }
    s = s - c[len-i]
  }

  return c;
}
```

4.6 CS-WOTS$^+$ Signature Generation (Function `wots_sign`)

A CS-WOTS$^+$ signature is a length `len` array of n-byte strings. The CS-WOTS$^+$ signature is first generated by mapping a message M to `len` integers between 0 and $w-1$, and the sum of them is equals to $\lfloor \texttt{len} \cdot (w-1)/2 \rfloor$. A CS-WOTS$^+$ hash address `ADRS`, a public seed `PK.seed`, and a secret seed `SK.seed` have to be provided by the calling algorithm. The address will encode the address of the CS-WOTS$^+$ key pair within a greater structure. Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of `ADRS` other than the last three 32-bit words. Note that the `PK.seed` used here is public information also available to a verifier while the secret seed `SK.seed` is private information. The pseudocode for generating a CS-WOTS$^+$ signature `SIG` is shown below (Algorithm 6).

The data format for a signature is given in Fig. 9.

Algorithm 6: `wots_sign` — Generating a CS-WOTS$^+$ signature on a message M.

```
# Input: Message M, secret seed SK.seed, public seed PK.seed, address ADRS
# Output: CS-WOTS+ signature sig
wots_sign(M, SK.seed, PK.seed, ADRS) {
  msg = wots_encode(M);
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    sig[i] = chain(sk, 0, msg[i], PK.seed, ADRS);
  }
  return sig;
}
```

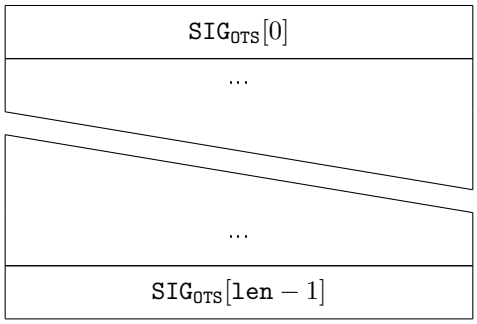


Figure 9: CS-WOTS$^+$ Signature Data Format.

4.7 CS-WOTS$^+$ Compute Public Key from Signature (Function `wots_pkFromSig`)

Kylin uses implicit signature verification for CS-WOTS$^+$. In order to verify a CS-WOTS$^+$ signature `SIG` on a message M, the verifier computes a CS-WOTS$^+$ public key value from the signature. This can be done by "completing" the chain computations starting from the signature values, using the base-w values of the message hash and its checksum. This step, called `wots_pkFromSig`, is described below in Algorithm 7. The result of `wots_pkFromSig` then has to be verified. In a standalone version, this would be done by simple comparison. When used in Kylin the output value is verified by using it to compute a Kylin public key.

A CS-WOTS$^+$ hash address `ADRS` and a public seed `PK.seed` have to be provided by the calling algorithm. The address will encode the address of the CS-WOTS$^+$ key pair within the Kylin structure. Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of `ADRS` other than the last three 32-bit words. Note that the `PK.seed` used here is public information also available to a verifier.

5 The Kylin Hypertree

In this section, we explain how the Kylin hypertree is built. We first explain how CS-WOTS$^+$ gets combined with a binary hash tree, leading to a fixed input-length version of the eXtended Merkle Signature Scheme (XMSS). Afterwards, we explain how to go to a hypertree from there. The hypertree might be viewed as a fixed input-length version of multi-tree XMSS (XMSS^{MT}).

Algorithm 7: `wots_pkFromSig` — Computing a CS-WOTS$^+$ public key from a message and its signature.

```
# Input: Message M, CS-WOTS+ signature sig, address ADRS, public seed PK.seed
# Output: CS-WOTS+ public key pk_sig derived from sig
wots_pkFromSig(sig, M, PK.seed, ADRS) {
  wotspkADRS = ADRS;
  msg = wots_encode(M)
  for ( i = 0; i < len; i++ ) {
    ADRS.setChainAddress(i);
    tmp[i] = chain(sig[i], msg[i], w - 1 - msg[i], PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk_sig = T_len(PK.seed, wotspkADRS, tmp);
  return pk_sig;
}
```

5.1 (Fixed Input-Length) XMSS

XMSS is a method for signing a potentially large but fixed number of messages. It is based on the Merkle signature scheme. It authenticates $2^{h_{\text{XMSS}}}$ CS-WOTS$^+$ public keys using a binary tree of height h_{XMSS}. Hence, an XMSS key pair for height h_{XMSS} can be used to sign 2 different messages. Each node in the binary tree is an n-byte value which is the tweakable hash of the concatenation of its two child nodes. The leaves are the CS-WOTS$^+$ public keys. The XMSS public key is the root node of the tree. In Kylin, the XMSS secret key is the single secret seed that is used to generate all CS-WOTS$^+$ secret keys.

An XMSS signature in the context of Kylin consists of the CS-WOTS$^+$ signature on the message and the so-called authentication path. The latter is a vector of tree nodes that allow a verifier to compute a value for the root of the tree starting from a CS-WOTS$^+$ signature. A verifier computes the root value and verifies its correctness. A standalone XMSS signature also contains the index of the used CS-WOTS$^+$ key pair. In the context of Kylin this is not necessary as the Kylin signature allows to compute the index for each XMSS signature contained.

5.1.1 XMSS Parameters

XMSS has the following parameters:

- h_{XMSS} : the height (number of levels $-$ 1) of the tree.
- n: the length in bytes of messages as well as of each node.
- w : the Winternitz parameter as defined for CS-WOTS$^+$ in the previous Section.

There are $2^{h_{\text{XMSS}}}$ leaves in the tree. XMSS signatures are denoted by $\texttt{SIG}_{\texttt{XMSS}}$ (`SIG_XMSS` in pseudocode). CS-WOTS$^+$ signatures are denoted by `SIG`.

XMSS parameters are implicitly included in algorithm inputs as needed.

5.1.2 XMSS Private Key

In the context of Kylin, an XMSS private key is the single secret seed `SK.seed` contained in the Kylin secret key. It is used to generate the CS-WOTS$^+$ secret keys within the structure of an XMSS key pair as described in Section 4.

5.1.3 TreeHash (Function `treehash`)

For the computation of the internal n-byte nodes of a Merkle tree, the subroutine `treehash` (Algorithm 8) accepts a secret seed `SK.seed`, a public seed `PK.seed`, an unsigned integer s (the start index), an unsigned integer z (the target node height), and an address `ADRS` that encodes the address of the containing tree. For the height of a node within a tree, counting starts with the leaves at height zero. The `treehash`

algorithm returns the root node of a tree of height z with the leftmost leaf being the CS-WOTS$^+$ pk at index s. It is REQUIRED that $s \,\%\, 2^z = 0$, i.e. that the leaf at index s is a leftmost leaf of a sub-tree of height z. Otherwise the algorithm fails as it would compute non-existent nodes. The `treehash` algorithm described here uses a stack holding up to $(z-1)$ nodes, with the usual stack functions `push()` and `pop()`. We furthermore assume that the height of a node (an unsigned integer) is stored alongside a node's value (an n-byte string) on the stack.

Algorithm 8: `treehash` — The TreeHash algorithm.

```
# Input: Secret seed SK.seed, start index s, target node height z, public seed
↪  PK.seed, address ADRS
# Output: n-byte root node - top node on Stack
treehash(SK.seed, s, z, PK.seed, ADRS) {
  if( s % (1 << z) != 0 ) return -1;
  for ( i = 0; i < 2^z; i++ ) {
    ADRS.setType(WOTS_HASH);
    ADRS.setKeyPairAddress(s + i);
    node = wots_PKgen(SK.seed, PK.seed, ADRS);
    ADRS.setType(TREE);
    ADRS.setTreeHeight(1);
    ADRS.setTreeIndex(s + i);
    while ( Top node on Stack has same height as node ) {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node = H(PK.seed, ADRS, (Stack.pop()  node));
      ADRS.setTreeHeight(ADRS.getTreeHeight() + 1);
    }
    Stack.push(node);
  }
  return Stack.pop();
}
```

5.1.4 XMSS Public Key Generation (Function `xmss_PKgen`)

The XMSS public key is computed as described in `xmss_PKgen` (Algorithm 9). In the context of Kylin the XMSS public key PK is the root of the binary hash tree. The root is computed using `treehash`. The public key generation takes a secret seed `SK.seed`, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of this XMSS instance within the Kylin structure.

Algorithm 9: `xmss_PKgen` — Generating an XMSS public key.

```
# Input: Secret seed SK.seed, public seed PK.seed, address ADRS
# Output: XMSS public key PK
xmss_PKgen(SK.seed, PK.seed, ADRS) {
  pk = treehash(SK.seed, 0, h_XMSS, PK.seed, ADRS)
  return pk;
}
```

5.1.5 XMSS Signature

An XMSS signature is a $((\texttt{len} + h_{\text{XMSS}}) \cdot n)$-byte string consisting of

- a CS-WOTS$^+$ signature $\texttt{SIG}_{\texttt{OTS}}$ taking $\texttt{len} \cdot n$ bytes,
- the authentication path `AUTH` for the leaf associated with the used CS-WOTS$^+$ key pair taking $h_{\text{XMSS}} \cdot n$ bytes.

The authentication path is an array of h_{XMSS} n-byte strings. It contains the siblings of the nodes in on the path from the used leaf to the root. It does not contain the nodes on the path itself. These nodes in `AUTH` are needed by a verifier to compute a root node for the tree from a CS-WOTS$^+$ public key. A node N is addressed by its position in the tree. $N(x, y)$ denotes the y-th node on level x with $y = 0$ being the leftmost node on a level. The leaves are on level 0, the root is on level h_{XMSS}. An authentication path contains exactly one node on every layer $0 \leq x \leq (h_{\text{XMSS}} - 1)$. For the i-th CS-WOTS$^+$ key pair, counting from zero, the j-th authentication path node is

$$\texttt{AUTH}[j] = N\left(j, \lfloor \frac{i}{2^j} \rfloor \oplus 1\right) \quad .$$

The computation of the authentication path is discussed in Section 5.1.6. The data format for a signature is given in Fig. 10.

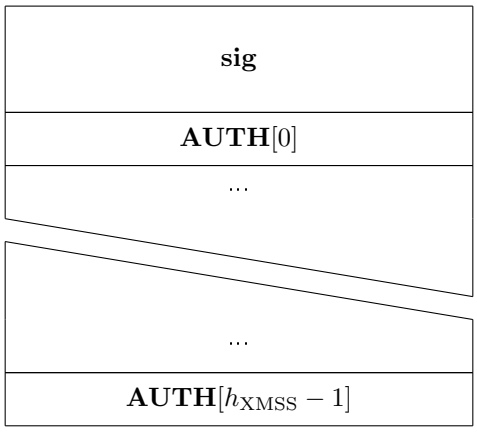


Figure 10: XMSS Signature

5.1.6 XMSS Signature Generation (Function `xmss_sign`)

To compute the XMSS signature of a message M in the context of Kylin, the secret seed `SK.seed`, the public seed `PK.seed`, the index `idx` of the CS-WOTS$^+$ key pair to be used, and the address `ADRS` of the XMSS instance are needed. First, a CS-WOTS$^+$ signature of the message digest is computed using the CS-WOTS$^+$ instance at index `idx`. Next, the authentication path is computed.

The node values of the authentication path MAY be computed in any way. The least memory-intensive method is to compute all nodes using the `treehash` algorithm (Algorithm 8). This is described here. Note that the details of how this step is implemented are not relevant to interoperability; it is not necessary to know any of these details in order to perform the signature verification operation.

5.1.7 XMSS Compute Public Key from Signature (Function `xmss_pkFromSig`)

Kylin makes use of implicit signature verification of XMSS signatures. An XMSS signature is used to compute a candidate XMSS public key, i.e., the root of the tree. This is used in further computations (signature of the tree above) and implicitly verified by the outcome of that computation. Hence, this specification does not contain an `xmss_verify` method but the method `xmss_pkFromSig`.

The method `xmss_pkFromSig` takes an n-byte message M, an XMSS signature $\texttt{SIG}_{\texttt{XMSS}}$, a signature index `idx`, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of the current XMSS instance within the virtual structure of the Kylin key pair. First, `wots_pkFromSig` is used to compute a candidate CS-WOTS$^+$ public key. This in turn is used together with the authentication path to compute a root node which is then returned. The algorithm `xmss_pkFromSig` is given as Algorithm 11.

5.2 HT: The Hypertree

The Kylin hypertree HT is a variant of XMSS^{MT}. It is essentially a certification tree of XMSS instances. A HT is a tree of several layers of XMSS trees. The trees on top and intermediate layers are used to sign the public keys, i.e., the root nodes, of the XMSS trees on the respective next layer below. Trees on the lowest layer are used to sign the actual messages, which are FORS public keys in Kylin.

Algorithm 10: `xmss_sign` — Generating an XMSS signature.

```
# Input: n-byte message M, secret seed SK.seed, index idx, public seed PK.seed,
  ↪ address ADRS
# Output: XMSS signature SIG_XMSS = (sig || AUTH)
xmss_sign(M, SK.seed, idx, PK.seed, ADRS) {
  // build authentication path
  for ( j = 0; j < h_XMSS; j++ ) {
    k = floor(idx / (2^j)) XOR 1;
    AUTH[j] = treehash(SK.seed, k * 2^j, j, PK.seed, ADRS);
  }
  ADRS.setType(WOTS_HASH);
  ADRS.setKeyPairAddress(idx);
  sig = wots_sign(M, SK.seed, PK.seed, ADRS);
  SIG_XMSS = sig  AUTH;
  return SIG_XMSS;
}
```

Algorithm 11: `xmss_pkFromSig` — Computing an XMSS public key from an XMSS signature.

```
# Input: index idx, XMSS signature SIG_XMSS = (sig || AUTH), n-byte message M,
  ↪ public seed PK.seed, address ADRS
# Output: n-byte root value node[0]
xmss_pkFromSig(idx, SIG_XMSS, M, PK.seed, ADRS){ // compute WOTS+ pk from WOTS+
  ↪ sig
ADRS.setType(WOTS_HASH);
ADRS.setKeyPairAddress(idx);
sig = SIG_XMSS.getWOTSSig();
AUTH = SIG_XMSS.getXMSSAUTH();
node[0] = wots_pkFromSig(sig, M, PK.seed, ADRS);
// compute root from WOTS+ pk and AUTH
ADRS.setType(TREE);
ADRS.setTreeIndex(idx);
for ( k = 0; k < h_XMSS; k++ ) {
  ADRS.setTreeHeight(k+1);
  if ( (floor(idx / (2^k)) % 2) == 0 ) {
      ADRS.setTreeIndex(ADRS.getTreeIndex() / 2);
      node[1] = H(PK.seed, ADRS, (node[0]  AUTH[k]));
    } else {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node[1] = H(PK.seed, ADRS, (AUTH[k]  node[0]));
    }
    node[0] = node[1];
  }
  return node[0];
}
```

Consider a HT of total height h that has d layers of XMSS trees with per-layer heights $(h_0, \ldots, h_{d-1})$ from bottom to top, where $\sum_{i=0}^{d-1} h_i = h$. Then layer $d-1$ contains one XMSS tree. In general, layer i contains $2^{\sum_{j=i+1}^{d-1} h_j}$ XMSS trees. Finally, layer 0 contains 2^{h-h_0} XMSS trees.

5.2.1 HT Parameters

In addition to all XMSS parameters, a HT requires the hypertree height h, the number of tree layers d, and per-layer XMSS heights $(h_0, \ldots, h_{d-1})$ with positive integers satisfying $\sum_{i=0}^{d-1} h_i = h$. The same Winternitz parameter w is used for all tree layers.

5.2.2 HT Key Generation (Function `ht_PKgen`)

The HT private key is the secret seed `SK.seed` which is used to generate all the CS-WOTS$^+$ private keys within the virtual structure spanned by the HT.

The HT public key is the public key (root node) of the single XMSS tree on the top layer. Its computation is explained below. The public key generation takes as input a private and a public seed.

Algorithm 12: `ht_PKgen` — Generating an HT public key.

```
# Input: Private seed SK.seed, public seed PK.seed
# Output: HT public key PK_HT
ht_PKgen(SK.seed, PK.seed){
      ADRS = toByte(0, 32);
      ADRS.setLayerAddress(d-1);
      ADRS.setTreeAddress(0);
      root = xmss_PKgen(SK.seed, PK.seed, ADRS);
      return root;
}
```

5.2.3 HT Signature

A HT signature $\texttt{SIG}_{\texttt{HT}}$ is a byte string of length $(h + d \cdot \texttt{len}) \cdot n$. It consists of d XMSS signatures, where layer i contributes one XMSS signature of $(h_i + \texttt{len}) \cdot n$ bytes.

The data format for a signature is given in Fig. 11.

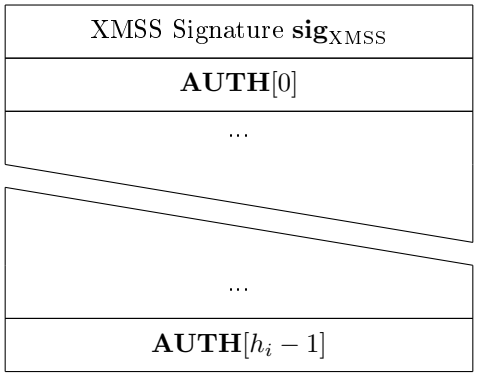


Figure 11: HT Signature.

5.2.4 HT Signature Generation (Function `ht_sign`)

To compute a HT signature $\texttt{SIG}_{\texttt{HT}}$ of a message M using, `ht_sign` (Algorithm 13) described below uses `xmss_sign` as defined in Section 5.1.6. The algorithm `ht_sign` takes as input a message M, a private seed `SK.seed`, a public seed `PK.seed`, and an index `idx`. The index identifies the leaf of the hypertree to be used to sign the message. The HT signature then consists of a stack of XMSS signatures using the XMSS trees on the path from the leaf with index idx to the top tree. Note that idx is passed as two

separate arguments, split into an index to address the specific tree and the leaf index within that tree. This allows for a somewhat higher hypertree, as one can use a 64-bit integer for `tree_idx` to support parameters that conform to $h < 64 + h_0$. This matches the parameters in this specification If other parameter sets are used that allow greater h, the data type of `tree_idx` MUST be adapted accordingly.

Algorithm `ht_sign` uses `xmss_pkFromSig` to compute the root node of an XMSS instance after that instance was used for signing. An alternative is to use `xmss_PKgen`. However, `xmss_PKgen` rebuilds the whole tree while `xmss_pkFromSig` only does one call to `wots_pkFromSig` and $(h_i - 1)$ calls to **H** on layer i. The algorithm `ht_sign` as described below is just one way to generate a HT signature. Other methods MAY be used as long as they generate the same output.

Algorithm 13: `ht_sign` — Generating an HT signature.

```
# Input: Message M, private seed SK.seed, public seed PK.seed, tree index
↪  idx_tree, leaf index idx_leaf
# Output: HT signature SIG_HT
ht_sign(M, SK.seed, PK.seed, idx_tree, idx_leaf) {
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  SIG_tmp = xmss_sign(M, SK.seed, idx_leaf, PK.seed, ADRS);
  SIG_HT = SIG_HT  SIG_tmp;
  root = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    SIG_tmp = xmss_sign(root, SK.seed, idx_leaf, PK.seed, ADRS);
    SIG_HT = SIG_HT  SIG_tmp;
    if ( j < d - 1 ) {
      root = xmss_pkFromSig(idx_leaf, SIG_tmp, root, PK.seed, ADRS);
    }
  }
  return SIG_HT;
}
```

5.2.5 HT Signature Verification (Function `ht_verify`)

HT signature verification (Algorithm 14) can be summarized as d calls to `xmss_pkFromSig` and one comparison with a given value. HT signature verification takes a message M, a signature $\texttt{SIG}_{\texttt{HT}}$, a public seed `PK.seed`, an index `idx` (split into a tree index and a leaf index, as above), and a HT public key $\texttt{PK}_{\texttt{HT}}$.

6 FORS: Forest Of Random Subsets

The Kylin hypertree HT is not used to sign the actual messages but the public keys of FORS instances which in turn are used to sign message digests. FORS, short for forest of random subsets, is a few-time signature scheme (FTS). FORS is an improvement of HORST [4] which in turn is a variant of HORS [26]. For security it is essential that the input to FORS is the output of a hash function. In the following we describe FORS as acting on bit strings.

FORS uses parameters k and $t = 2^a$ (example parameters are $t = 2^{15}, k = 10$). FORS signs strings of length ka bits. Here, we deviate from defining sizes in bytes as the message length in bits might not be a multiple of eight. The private key consists of kt random n- byte strings grouped into k sets, each containing tn-byte strings. The private key values are pseudorandomly generated from the main private

Algorithm 14: `ht_verify` — Verifying a HT signature $\mathbf{SIG_{HT}}$ on a message M using a HT public key $\mathbf{PK}_{HT}$.

```
# Input: Message M, signature SIG_HT, public seed PK.seed, tree index idx_tree,
↪  leaf index idx_leaf, HT public key PK_HT.
# Output: Boolean
ht_verify(M, SIG_HT, PK.seed, idx_tree, idx_leaf, PK_HT){
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // verify
  SIG_tmp = SIG_HT.getXMSSSignature(0);
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  node = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    SIG_tmp = SIG_HT.getXMSSSignature(j);
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    node = xmss_pkFromSig(idx_leaf, SIG_tmp, node, PK.seed, ADRS);
    }
  if ( node == PK_HT ) {
    return true;
  } else {
    return false;
  }
}
```

seed `SK.seed` in the Kylin private key. In Kylin, the FORS private key values are only temporarily generated as an intermediate result when computing the public key or a signature.

The FORS public key is a single n-byte hash value. It is computed as the tweakable hash of the root nodes of k binary hash trees. Each of these binary hash trees has height a and is used to authenticate the t private key values of one of the k sets. Accordingly, the leaves of a tree are the (tweakable) hashes of the values in its private key set.

A signature on a string M consists of k private key values — one per set of private key elements — and the associated authentication paths. To compute the signature, `md` is split into k a-bit strings. As `md` is a sequence of bytes, we first convert to a bit-string by enumerating the bytes in `md`, internally enumerating the bits within a byte from least to most significant. Next, each of these bit strings is interpreted as an integer between 0 and $t-1$. Each of these integers is used to select one private key value from a set. I.e., if the first integer is i, the i-th private key element of the first set gets selected and so on. The signature consists of the selected private key elements and the associated authentication paths.

Kylin uses implicit verification for FORS, only using a method to compute a candidate public key from a signature. This is done by computing root nodes of the k trees using the indices computed from the input string as well as the private key values and authentication paths form the signature. The tweakable hash of these roots is then returned as candidate public key.

We now describe the parameters and algorithms for FORS.

6.1 FORS Parameters

FORS uses the parameters n, k, and t; they all take positive integer values. These parameters are summarized as follows:

- n: the security parameter; it is the length of a private key, public key, or signature element in bytes.
- k: the number of private key sets, trees and indices computed from the input string.
- t: the number of elements per private key set, number of leaves per hash tree and upper bound on the index values. The parameter t MUST be a power of 2. If $t = 2^a$, then the trees have height a and the input string is split into bit strings of length a.

Inputs to FORS are bit strings of length $k \log t$.

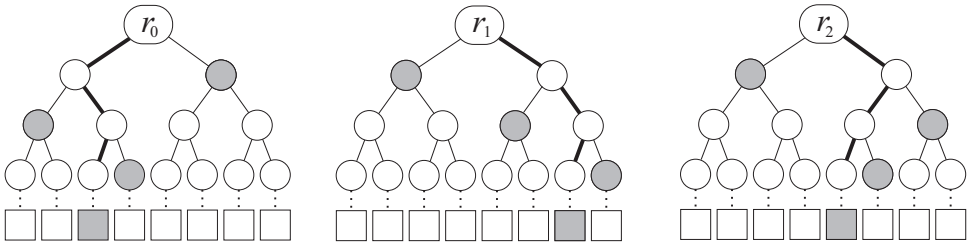


Figure 12: FORS trees and PK.

6.2 FORS Private Key (Function `fors_SKgen`)

In the context of Kylin, a FORS private key is the single private seed `SK.seed` contained in the Kylin private key. It is used to generate the kt n-byte private key values using **PRF** with a FORS key generation address. While these values are logically grouped into a two-dimensional array, for implementations it makes sense to assume they are in a one dimensional array of length kt. The jth element of the i-th set is then stored at `SK`$[it + j]$. To generate one of these elements, a FORS key generation address skADRS is used, that encodes the position of the FORS key pair within Kylin and has tree height set to 0 and leaf index set to $it + j$:

Algorithm 15: `fors_SKgen` — Computing a FORS private key value.

```
# Input: secret seed SK.seed, address ADRS, secret key index idx = it+j
# Output: FORS private key sk
fors_SKgen(SK.seed, ADRS, idx) {
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(FORS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());

  skADRS.setTreeHeight(0);
  skADRS.setTreeIndex(idx);
  sk = PRF(SK.seed, skADRS);
  return sk;
}
```

6.3 FORS TreeHash (Function `fors_treehash`)

Before coming to the FORS public key, we have to discuss computation of the trees. For the computation of the n-byte nodes in the FORS hash trees, the subroutine `fors_treehash` is used. It is essentially the same algorithm as `treehash` (Algorithm 8) in Section 5.1. The two differences are how the leaf nodes are computed and how addresses are handled. However, as the addresses are similar, an implementation can implement both algorithms in the same routine easily.

Algorithm `fors_treehash` accepts a secret seed `SK.seed`, a public seed `PK.seed`, an unsigned integer s (the start index), an unsigned integer z (the target node height), and an address `ADRS` that encodes the address of the FORS key pair. As for `treehash`, the `fors_treehash` algorithm returns the root node of a tree of height z with the leftmost leaf being the hash of the private key element at index s. Here, s is ranging over the whole kt private key elements. It is REQUIRED that $s \,\%\, 2^z = 0$, i.e. that the leaf at index s is a leftmost leaf of a sub-tree of height z. Otherwise the algorithm fails as it would compute non-existent nodes.

Algorithm 16: The `fors_treehash` algorithm.

```
# Input: Secret seed SK.seed, start index s, target node height z, public seed
↪  PK.seed, address ADRS
# Output: n-byte root node - top node on Stack
fors_treehash(SK.seed, s, z, PK.seed, ADRS) {
  if( s % (1 << z) != 0 ) return -1;
  for ( i = 0; i < 2^z; i++ ) {
    sk = fors_SKgen(SK.seed, ADRS, s+i)
    node = F(PK.seed, ADRS, sk);
    ADRS.setTreeHeight(1);
    ADRS.setTreeIndex(s + i);
    while ( Top node on Stack has same height as node ) {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node = H(PK.seed, ADRS, (Stack.pop()  node));
      ADRS.setTreeHeight(ADRS.getTreeHeight() + 1);
    }
    Stack.push(node);
  }
  return Stack.pop();
}
```

6.4 FORS Public Key (Function `fors_PKgen`)

In the context of Kylin, the FORS public key is never generated alone. It is only generated together with a signature. We include `fors_PKgen` for completeness, a better understanding, and testing. Algorithm `fors_PKgen` takes a private seed `SK.seed`, a public seed `PK.seed`, and a FORS address `ADRS`. The latter encodes the position of the FORS instance within Kylin. It outputs a FORS public key.

Algorithm 17: `fors_PKgen` — Generate a FORS public key.

```
# Input: Secret seed SK.seed, public seed PK.seed, address ADRS
# Output: FORS public key PK
fors_PKgen(SK.seed, PK.seed, ADRS) {
  forspkADRS = ADRS; // copy address to create FTS public key address
  for(i = 0; i < k; i++){
    root[i] = fors_treehash(SK.seed, i*t, a, PK.seed, ADRS);
  }
  forspkADRS.setType(FORS_ROOTS);
  forspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_k(PK.seed, forspkADRS, root);
  return pk;
}
```

6.5 FORS Signature Generation (Function `fors_sign`)

A FORS signature is a length $k(\log t+1)$ array of n-byte strings. It contains k private key values, n-bytes each, and their associated authentication paths, $\log t$ n-byte values each.

The algorithm `fors_sign` takes a $(k \log t)$-bit string M, a private seed `SK.seed`, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of the FORS instance within Kylin. It outputs a FORS signature $\mathtt{SIG}_{\mathtt{FORS}}$.

Algorithm 18: `fors_sign` — Generate a FORS signature on string M.

```
# Input: Bit string M, secret seed SK.seed, address ADRS, public seed PK.seed
# Output: FORS signature SIG_FORS
fors_sign(M, SK.seed, PK.seed, ADRS) {
  // compute signature elements
  for(i = 0; i < k; i++){
    // get next index
    unsigned int idx = bits i*log(t) to (i+1)*log(t) - 1 of M;
    // pick private key element
    SIG_FORS = SIG_FORS  fors_SKgen(SK.seed, ADRS, i*t + idx) ;
    // compute auth path
    for ( j = 0; j < a; j++ ) {
      s = floor(idx / (2^j)) XOR 1;
      AUTH[j] = fors_treehash(SK.seed, i * t + s * 2^j, j, PK.seed, ADRS);
    }
    SIG_FORS = SIG_FORS  AUTH;
  }
  return SIG_FORS;
}
```

The data format for a signature is given in Fig. 13.

6.6 FORS Compute Public Key from Signature (Function `fors_pkFromSig`)

Kylin makes use of implicit signature verification of FORS signatures. A FORS signature is used to compute a candidate FORS public key. This public key is used in further computations (message for the

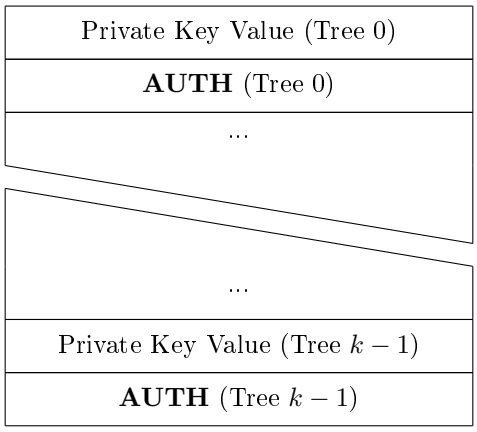


Figure 13: FORS Signature.

signature of the XMSS tree above) and implicitly verified by the outcome of that computation. Hence, this specification does not contain a `fors_verify` method but the method `fors_pkFromSig`.

The method `fors_pkFromSig` takes a $k \log t$-bit string M, a FORS signature $\texttt{SIG}_{\texttt{FORS}}$, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of the FORS instance within the virtual structure of the Kylin key pair. First, the roots of the k binary hash trees are computed using `fors_treehash`. Afterwards the roots are hashed using the tweakable hash function T_k. The algorithm `fors_pkFromSig` is given as Algorithm 19. The method `fors_pkFromSig` makes use of functions $\texttt{SIG}_{\texttt{FORS}}$`.getSK(i)` and $\texttt{SIG}_{\texttt{FORS}}$`.getAUTH(i)`. The former returns the i-th secret key element stored in the signature, the latter returns the i-th authentication path stored in the signature.

7 The Full Kylin Signature Scheme

We now have all ingredients to describe our main construction Kylin. Essentially, Kylin follows the fundamental structure of Kylin with the only difference being the one-time signature component.

7.1 Kylin Parameters

Kylin has the following parameters:

- n: the security parameter in bytes.
- w: the Winternitz parameter as defined in Section 4.1.
- h: the height of the hypertree as defined in Section 5.2.1.
- d: the number of layers in the hypertree as defined in Section 5.2.1.
- k: the number of trees in FORS as defined in Section 6.1.
- t: the number of leaves of a FORS tree as defined in Section 6.1.

All the restrictions stated in the previous sections apply. Recall that we use $a = \log t$. Moreover, from these values the values m and `len` are computed as

- m: the message digest length in bytes. It is computed as

$$m = \lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

  While only $h + k \log t$ bits would be needed, using the longer m as defined above simplifies implementations significantly.

Algorithm 19: `fors_pkFromSig` — Compute a FORS public key from a FORS signature.

```
# Input: FORS signature SIG_FORS, (k lg t)-bit string M, public seed PK.seed,
↪  address ADRS
# Output: FORS public key
fors_pkFromSig(SIG_FORS, M, PK.seed, ADRS){
  // compute roots
  for(i = 0; i < k; i++){
    // get next index
    unsigned int idx = bits i*log(t) to (i+1)*log(t) - 1 of M;
    // compute leaf
    sk = SIG_FORS.getSK(i);
    ADRS.setTreeHeight(0);
    ADRS.setTreeIndex(i*t + idx);
    node[0] = F(PK.seed, ADRS, sk);
    // compute root from leaf and AUTH
    auth = SIG_FORS.getAUTH(i);
    ADRS.setTreeIndex(i*t + idx);
    for ( j = 0; j < a; j++ ) {
      ADRS.setTreeHeight(j+1);
      if ( (floor(idx / (2^j)) % 2) == 0 ) {
        ADRS.setTreeIndex(ADRS.getTreeIndex() / 2);
        node[1] = H(PK.seed, ADRS, (node[0]  auth[j]));
      } else {
        ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
        node[1] = H(PK.seed, ADRS, (auth[j]  node[0]));
      }
      node[0] = node[1];
    }
    root[i] = node[0];
  }
  forspkADRS = ADRS; // copy address to create FTS public key address
  forspkADRS.setType(FORS_ROOTS);
  forspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_k(PK.seed, forspkADRS, root);
  return pk;
}
```

- `len`: the number of n-byte string elements in a CS-WOTS$^+$ private key, public key, and signature. The value `len` is chosen so that the size of the following poset is larger than the desired message space $\{0,1\}^n$ (cf. Section 2).

$$\left| D_{\texttt{len},\lfloor \frac{\texttt{len}\cdot(w-1)}{2} \rfloor} \right| \geq 2^n \ .$$

In the following, we assume that all algorithms have access to these parameters.

7.2 Kylin Key Generation (Function `kylin_keygen`)

The Kylin private key contains two elements. First, the n-byte secret seed `SK.seed` which is used to generate all the CS-WOTS$^+$ and FORS private key elements. Second, an n-byte PRF key `SK.prf` which is used to deterministically generate a randomization value for the randomized message hash.

The Kylin public key also contains two elements. First, the HT public key, i.e. the root of the tree on the top layer. Second, an n-byte public seed value `PK.seed` which is sampled uniformly at random.

As `kylin_sign` does not get the public key, but needs access to `PK.seed` (and possibly to `PK.root` for fault attack mitigation), the Kylin secret key contains a copy of the public key.

The description of algorithm `kylin_keygen` assumes the existence of a function `sec_rand` which on input i returns i-bytes of cryptographically strong randomness.

Algorithm 20: `kylin_keygen` — Generate a Kylin key pair.

```
# Input: (none)
# Output: Kylin key pair (SK,PK)
kylin_keygen( ){
  SK.seed = sec_rand(n);
  SK.prf = sec_rand(n);
  PK.seed = sec_rand(n);
  PK.root = ht_PKgen(SK.seed, PK.seed);
  return ( (SK.seed, SK.prf, PK.seed, PK.root), (PK.seed, PK.root) );
}
```

The format of a Kylin private and public key is given in Fig. 14.

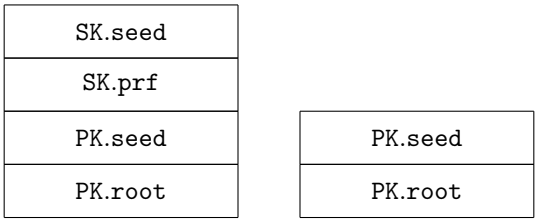


Figure 14: Left: Kylin secret key. Right: Kylin public key.

7.3 Kylin Signature

A Kylin signature $\texttt{SIG}_{\texttt{HT}}$ is a byte string of length $(1 + k \cdot (a+1) + h + d \cdot \texttt{len})n$. It consists of an n-byte randomization string `R`, a FORS signature $\texttt{SIG}_{\texttt{FORS}}$ consisting of $k(a+1)n$-byte strings, and a HT signature $\texttt{SIG}_{\texttt{HT}}$ of $(h + d \cdot \texttt{len})n$ bytes.

The data format for a signature is given in Fig. 15.

7.4 Kylin Signature Generation (Function `kylin_sign`)

Generating a Kylin signature consists of four steps. First, a random value `R` is pseudorandomly generated. Next, this is used to compute a m byte message digest which is split into a $\lfloor (k \log t + 7)/8 \rfloor$-byte partial message digest `tmp_md`, a $\lfloor (h - h_0 + 7)/8 \rfloor$-byte tree index `tmp_idx_tree`, and a $\lfloor (h_0 + 7)/8 \rfloor$-byte leaf index `tmp_idx_leaf`. Next, the actual values `md`, `idx_tree`, and `idx_leaf` are computed by extracting the necessary number of bits. The partial message digest md is then signed with the `idx_leaf`-th FORS

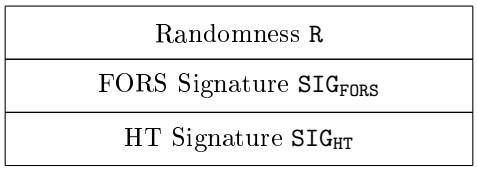


Figure 15: Kylin signature.

key pair of the idx_tree-th XMSS tree on the lowest HT layer. The public key of the FORS key pair is then signed using HT.The index is never actually used as a whole, but immediately split into a tree index and a leaf index, for ease of implementation.

When computing R, the PRF takes a n-byte string opt which is initialized with PK.seed but can be overwritten with randomness if the global variable RANDOMIZE is set. This option is given as otherwise Kylin signatures would be always deterministic.

Algorithm 21: kylin_sign — Generating a Kylin signature.

```
# Input: Message M, private key SK = (SK.seed, SK.prf, PK.seed, PK.root)
# Output: Kylin signature SIG
kylin_sign(M, SK){ // init
ADRS = toByte(0, 32);
  // generate randomizer
  opt = PK.seed;
  if(RANDOMIZE){
    opt = rand(n);
  }
  R = PRF_msg(SK.prf, opt, M);
  SIG = SIG  R;
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  tmp_md = first floor((ka +7)/ 8) bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  md = first ka bits of tmp_md;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // FORS sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(FORS_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  SIG_FORS = fors_sign(md, SK.seed, PK.seed, ADRS);
  SIG = SIG  SIG_FORS;
  // get FORS public key
  PK_FORS = fors_pkFromSig(SIG_FORS, md, PK.seed, ADRS);
  // sign FORS public key with HT
  ADRS.setType(TREE);
  SIG_HT = ht_sign(PK_FORS, SK.seed, PK.seed, idx_tree, idx_leaf);
  SIG = SIG  SIG_HT;
  return SIG;
}
```

7.5 Kylin Signature Verification (Function kylin_verify)

Kylin signature verification (Algorithm 22) can be summarized as recomputing message digest and index, computing a candidate FORS public key, and verifying the HT signature on that public key. Note that the

HT signature verification will fail if the FORS public key is not matching the real one (with overwhelming probability). Kylin signature verification takes a message M, a signature `SIG`, and a Kylin public key `PK`.

Algorithm 22: `kylin_verify` — Verify a Kylin signature **SIG** on a message M using a Kylin public key **PK**.

```
# Input: Message M, signature SIG, public key PK
# Output: Boolean
kylin_verify(M, SIG, PK){ // init
  ADRS = toByte(0, 32);
  R = SIG.getR();
  SIG_FORS = SIG.getSIG_FORS();
  SIG_HT = SIG.getSIG_HT();
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  tmp_md = first floor((ka +7)/ 8) bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  md = first ka bits of tmp_md;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // compute FORS public key
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(FORS_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  PK_FORS = fors_pkFromSig(SIG_FORS, md, PK.seed, ADRS);
  // verify HT signature
  ADRS.setType(TREE);
  return ht_verify(PK_FORS, SIG_HT, PK.seed, idx_tree, idx_leaf, PK.root);
}
```

8 Parameter Sets and Hash/XOF Instantiation

This section discusses instantiations for Kylin. Kylin can be viewed as a signature template. It is a way to build a signature scheme by instantiating the cryptographic function families used. We consider different ways to implement the cryptographic function families as different signature systems. Orthogonal to instantiating the cryptographic function families are parameter sets. Parameter sets assign specific values to the Kylin parameters described below.

In this section, we first define the requirements on parameters and discuss existing trade-offs between security, sizes, and speed controlled by the different parameters. Then we summarize the frozen NGCC-facing parameter sets and the hash/XOF instantiation used by the packaged implementation.

8.1 Kylin Parameter Sets

Kylin is described by the following parameters already described in the previous sections. All parameters take positive integer values.

- n: the security parameter in bytes.
- w : the Winternitz parameter.
- h: the height of the hypertree.
- d: the number of layers in the hypertree.
- k: the number of trees in FORS.

- t: the number of leaves of a FORS tree.

Recall that we use $a = \log t$. Moreover, from these values the values m and `len` are computed as

- m: the message digest length in bytes. It is computed as

$$m = \lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

- `len`: the number of n-byte string elements in a CS-WOTS$^+$ private key, public key, and signature.

$$\texttt{len} = \min \left\{ \ell : \left| D_{\ell, \lfloor \frac{\ell \cdot (w-1)}{2} \rfloor} \right| \geq 2^n \right\} \ ,$$

  where the size of $D_{\ell, \lfloor \frac{\ell \cdot (w-1)}{2} \rfloor}$ is evaluated using the recurrence relation in Eq. (1).

We now repeat the roles of, requirements on, and properties of these parameters. Afterwards, we give several formulas that show their exact influence on performance and security. The security parameter n is also the output length of all cryptographic function families besides $\mathbf{H_{msg}}$. Therefore, it largely determines which security level a parameter set reaches. It is also the size of virtually any node within the Kylin structure and thereby also the size of all elements in a signature, i.e., the signature size is a multiple of n.

The Winternitz parameter w determines the number and length of the hash chains per CS-WOTS$^+$ instance. A greater value for w linearly increases the length of the hash chains but logarithmically reduces their number. The number of hash chains exactly corresponds to the number of n-byte values in a CS-WOTS$^+$ signature. Thereby it largely influences the size of a Kylin signature. The product of the number and the length of hash chains directly correlates with signing speed as essentially all time in HT signature generation is spent computing CS-WOTS$^+$ public keys. Therefore, greater w means shorter signatures but slower signing. However, note the exponential gap. The bigger w gets, the more expensive is the signature size reduction. The Winternitz parameter does not influence Kylin security.

The height of the hypertree h determines the number of FORS instances. Hence, it determines the probability that a FORS key pair is used several times, given the number of signatures made with a Kylin key pair. Hence, the height has a direct impact on security: A taller hypertree gives more security. On the other hand, a taller tree leads to larger signatures.

The number of layers d is a pure performance trade-off parameter and does not influence security. It determines the number of layers of XMSS trees in the hypertree. The per-layer heights $(h_0, \ldots, h_{d-1})$ define the XMSS heights and must satisfy $\sum_{i=0}^{d-1} h_i = h$. Increasing d usually allows smaller subtrees and faster signing. However, d also controls the number of layers and thereby the number of CS-WOTS$^+$ signatures within a HT and thereby a Kylin signature.

The parameters k and t determine the performance and security of FORS. The number of leaves of a tree in FORS t must be a power of two while k can be chosen freely. A smaller t generally leads to smaller and faster signatures. However, for a given security level a smaller t requires a greater k which increases signature size and slows down signing. Hence, it is important to balance these two parameters. This is best done using the formulas below.

The message digest length m is the output length of $\mathbf{H_{msg}}$ in bytes. It is $\lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor$ bytes.

The number `len` of chains in a CS-WOTS$^+$ key pair determines the CS-WOTS$^+$ signature size.

8.1.1 Influence of Parameters on Security and Performance

In the following we provide formulas to compute speed, size and security for a given Kylin parameter set. This supports parameter selection. We also provide a SAGE script in Appendix A.

Key Generation. Generating the Kylin private key and `PK.seed` requires three calls to a secure random number generator. Next we have to generate the top tree (layer $d-1$). For the leaves we need to do $2^{h_{d-1}}$ CS-WOTS$^+$ key generations (`len` calls to PRF for generating the `SK` and w`len` calls to $\mathbf{F}$ for the `PK`) and we have to compress the CS-WOTS$^+$ public key (one call to $\mathbf{T}_{\texttt{len}}$). Computing the root of the top tree requires $(2^{h_{d-1}} - 1)$ calls to $\mathbf{H}$.

Signing. For randomization and message compression we need one call to $\mathbf{PRF_{msg}}$, and one to $\mathbf{H_{msg}}$. The FORS signature requires kt calls to **PRF** and **F**. Further, we have to compute the root of k binary trees of height $\log t$ which adds $k(t-1)$ calls to **H**. Finally, we need one call to $\mathbf{T}_k$. Next, we compute one HT signature which consists of d trees similar to the key generation. Hence, we have to do $\sum_{i=0}^{d-1} 2^{h_i}$ times `len` calls to **PRF** and w`len` calls to **F** as well as $\sum_{i=0}^{d-1} 2^{h_i}$ calls to $\mathbf{T}_{\texttt{len}}$. For computing the root of each tree we get additionally $\sum_{i=0}^{d-1}(2^{h_i}-1)$ calls to **H**.

Verification. First we need to compute the message hash using $\mathbf{H_{msg}}$. We need to do one FORS verification which requires k calls to **F** (to compute the leaf nodes from the signature elements), $k \log t$ calls to **H** (to compute the root nodes using the leaf nodes and the authentication paths), and one call to $\mathbf{T}_k$ for hashing the roots. Next, we have to verify d XMSS signatures which takes $< w$`len` calls to **F** and one call to $\mathbf{T}_{\texttt{len}}$ each for CS-WOTS$^+$ signature verification[2]. It also needs $\sum_{i=0}^{d-1} h_i = h$ calls to **H** for the d root computations.

The size of the Kylin private and public keys along with the signature are $4n, 2n, (h + k(a+1) + d \cdot \texttt{len} + 1)n$ respectively.

The classical security level, or bit security of Kylin against generic attacks can be computed as

$$\texttt{bitsec} = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^{h}}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) \ .$$

The quantum security level, or bit security of Kylin against generic attacks can be computed as

$$\texttt{bitsec} = -\frac{1}{2}\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^{h}}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) \ .$$

Here, we are neglecting the small constant factors inside the logarithm.

8.1.2 Proposed Parameter Sets and Security Levels

As explained in the previous subsection, even for a fixed security level the design of Kylin supports different tradeoffs between signature size and speed. For this submission package, we use the three NGCC-aligned Kylin instances listed in Table 3. These instances are generated by the repository-local parameter script and tuned for the call's required 128/256/512-bit classical levels together with the corresponding quantum targets.

Table 3: Frozen NGCC-aligned Kylin parameter families used by the packaged primary implementations.

Family	NGCC lvl	q	n	h/d	$(h_0, \ldots, h_{d-1})$	a	k	w	`len`	bitsec	pk	sk	sig
Kylin-128	128/80	2^{64}	16	65/9	(7, 7, 7, 7, 7, 7, 7, 8, 8)	13	11	73	22	127	32	64	6688
Kylin-256	256/128	2^{64}	32	68/10	(6, 6, 7, 7, 7, 7, 7, 7, 7, 7)	13	22	54	46	256	64	128	26784
Kylin-512	512/256	2^{64}	64	66/11	(6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6)	13	49	97	79	511	128	256	103808

8.2 Instantiations of Hash Functions

For the NGCC submission package, hash and XOF calls follow the ICCS auxiliary interface used by the bundled implementation code. In particular, the current package uses `sm3hash`, `pseudohash`, and `pseudoXOF`, with domain-separated wrappers built on top of `pseudoXOF` in the `hash_iccs.c` and `thash_iccs.c` modules.

Recall that n and m are the security parameter and the message-digest length in bytes.

[2]It should be noted that the w`len` bound for calls to **F** is a worst-case bound. This is a bound on the cost for CS-WOTS$^+$ signature verification. Given that the messages are hash values which can assumed to be close to uniformly distributed, this value will be closer to the average-case bound $(w/2)$. `len` in actual measurements.

The implementation-level mapping is summarized as follows:

$$\begin{aligned}
\mathbf{H_{msg}}(\mathtt{R}, \mathtt{PK.seed}, \mathtt{PK.root}, M) &= \text{pseudoXOF}(\mathtt{tag}_{H_msg} \| \mathtt{R} \| \mathtt{PK.seed} \| \mathtt{PK.root} \| M, m),\\
\mathbf{PRF}(\mathtt{PK.seed}, \mathtt{SK.seed}, \mathtt{ADRS}) &= \text{pseudoXOF}(\mathtt{tag}_{PRF} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| \mathtt{SK.seed}, n),\\
\mathbf{PRF_{msg}}(\mathtt{SK.prf}, \mathtt{OptRand}, M) &= \text{pseudoXOF}(\mathtt{tag}_{PRFmsg} \| \mathtt{SK.prf} \| \mathtt{OptRand} \| M, n),\\
\mathbf{F}(\mathtt{PK.seed}, \mathtt{ADRS}, M_1) &= \text{pseudoXOF}(\mathtt{tag}_{F} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M_1, n),\\
\mathbf{H}(\mathtt{PK.seed}, \mathtt{ADRS}, M_1 \| M_2) &= \text{pseudoXOF}(\mathtt{tag}_{H} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M_1 \| M_2, n),\\
\mathbf{T}_\ell(\mathtt{PK.seed}, \mathtt{ADRS}, M) &= \text{pseudoXOF}(\mathtt{tag}_{T} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M, n).
\end{aligned}$$

Here, `sm3hash` is the fixed-output SM3 interface and `pseudohash`/`pseudoXOF` provide the variable-length hash and XOF behavior required by the NGCC API profile. The public packaged implementation uses explicit domain-separation tags for these calls and keeps the address/seed layout aligned with the ICCS helper functions.

9 Design Rationale

The design rationale behind Kylin is to follow the original SPHINCS$^+$ construction, apply the optimized CS-WOTS$^+$ one-time signature scheme, and use a Diverse Layer Heights optimization for the hypertree. The idea behind SPHINCS$^+$ was as follows. One can build a stateless hash-based signature scheme using a massive binary certification tree and selecting a leaf at random for each message to be signed. The problem with this approach is that the tree has to be extremely high, i.e., a height of about twice the security level would be necessary. This leads to impractical signature sizes. Using a hypertree instead of a binary certification tree allows the design to trade speed for signature size. However, this is still not sufficient by itself to get practical sizes and speed.

Mathematical hard problem. Kylin is a stateless hash-based post-quantum digital signature scheme. Its security is inherited from the hardness of generic attacks against the underlying hash and XOF primitives, following the SPHINCS$^+$ proof framework. At the construction level, Kylin can be viewed as an improvement of the NIST-standardized SPHINCS+ design: it keeps the hash-based hypertree and FORS structure and replaces the Winternitz one-time signature layer with CS-WOTS$^+$.

Parameter selection criteria. The primary parameter-selection rule is to improve on comparable SPHINCS$^+$-style profiles while keeping the implementation and analysis close to the original construction. Under the fixed NGCC security levels, the search gives priority to smaller signatures, then faster verification, and then faster signing, while keeping the one-key signing budget at $q = 2^{64}$. The selected instances in Table 3 therefore use three main design levers: the constant-sum CS-WOTS$^+$ encoding, the FORS parameters (a, k), and the per-layer hypertree profile $(h_0, \ldots, h_{d-1})$.

Since one-time signatures are used extensively in the SPHINCS$^+$ hypertree structure, optimizing the one-time signature size has a direct influence on the overall performance. With CS-WOTS$^+$, Kylin can retune the signature scheme to obtain smaller signatures while preserving comparable running time. Diverse Layer Heights further removes the unnecessary restriction that h be divisible by d: the layers may have different heights as long as their sum is h, which lets the search use balanced profiles such as $(7, 7, 7, 7, 7, 7, 8, 8)$ for `Kylin-128`.

Failure probability and security impact. Kylin has no decryption- or decapsulation-style correctness failure event. For honestly generated keys and honestly generated signatures, verification is deterministic and accepts except for implementation faults or violation of the underlying cryptographic assumptions. The probability term used in the parameter search is therefore a forgery-success bound, not a correctness-failure probability.

For a signing budget of q signatures under one key pair, the concrete parameter-search exponent is derived from

$$b = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1 - \frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right).$$

The selected primary instances set $q = 2^{64}$ and give the bit-security exponents shown in Table 3. Thus the relevant probability term is bounded by approximately $2^{-127.209}$, $2^{-255.944}$, and $2^{-511.243}$ for the

three primary instances respectively. These exponents are used to judge the impact of repeated FORS index selection and generic forgery events on the claimed security levels.

Other considerations. The repository also maintains $q = 2^{24}$ companion profiles as reference-only additional implementations. These profiles are not the primary NGCC submission instances, but they are useful for applications with a smaller per-key signing budget and stricter signature-size preferences; see Appendix B.

We are able to prove that the encoding scheme in CS-WOTS$^+$ is size optimal among all encoding schemes over tree structures, even including those not necessarily efficiently computable. Moreover, to the best of our knowledge, no existing DAG-based scheme achieves better performance in terms of signature size, which renders our scheme the size-optimal one among all known schemes [30]. We are aware of probabilistic encoding schemes based on rejection sampling [18], and we consider those techniques to be of independent interest.

10 Security Statements and Analyses

This section serves as the security-claim section of the submission draft. For the digital-signature functionality specified in Section 7, we claim EUF-CMA security under the assumptions and instantiations discussed below. The recommended NGCC-aligned families are recorded in Table 3, and the packaged ICCS hash/XOF interface is described in Section 8.2.

Theoretical security. Kylin follows the security-analysis framework of SPHINCS$^+$. As a result, the EUF-CMA security of the signature scheme is intended to reduce to the security of the tweakable hash-function families used for $\mathbf{H_{msg}}$, $\mathbf{PRF}$, $\mathbf{PRF_{msg}}$, $\mathbf{F}$, $\mathbf{H}$, and $\mathbf{T}$. In the usual analysis model, these primitives are treated as idealized random oracles or quantum-access random oracles, and the resulting bounds are interpreted against generic classical and quantum attacks.

In particular, the EUF-CMA security of the signature scheme relies on the following security properties of the underlying components:

- The PRF security of $\mathbf{PRF}$ and $\mathbf{PRF_{msg}}$,
- The interleaved target subset resilience (ITSR) of $\mathbf{H_{msg}}$,
- The single function, multi-target undetectability (SM-UD), target-collision (SM-TCR), and decisional second-preimage resistance (SM-DSPR) of $\mathbf{F}$,
- The SM-TCR of $\mathbf{H}$ and $\mathbf{T}_\ell$.

Concrete security and failure probability. The security analysis is identical to SPHINCS$^+$. The classical security level, or bit security of Kylin against generic attacks can be computed as

$$b = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

The quantum security level, or bit security of Kylin against generic attacks can be computed as

$$b = -\frac{1}{2}\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

Here, we are neglecting the small constant factors inside the logarithm.

For the recommended NGCC submission families in Table 3, we instantiate these formulas with $q = 2^{64}$ in order to match the call's explicit requirement that one key pair support at least 2^{64} signed messages.

Other security properties. No additional formal security claims are made in this draft. In particular, the current first-stage text does not claim dedicated side-channel resistance, perfect forward security, a quantified multi-key reduction, or a separate security theorem for temporary key reuse beyond the stated single-key $q = 2^{64}$ analysis.

Security of the ICCS hash/XOF instantiation. The packaged NGCC implementation uses ICCS auxiliary functions `sm3hash`, `pseudohash`, and `pseudoXOF`. At the scheme level, $\mathbf{H_{msg}}$, **PRF**, $\mathbf{PRF_{msg}}$, **F**, **H**, and **T** are realized through domain-separated wrappers over `pseudoXOF` with explicit tags and seed/address separation in the input layout.

Accordingly, the security claim in this draft is that the concrete instantiated functions provide the pseudorandomness, target-collision resistance, and related properties required by the Kylin proof framework at the selected parameter sizes. This submission text does not introduce an additional independent tightness claim specific to the ICCS helper internals beyond that standard generic-analysis interpretation.

11 Performance Evaluation

This section provides the performance analysis and benchmark data required by the call. The current repository uses repository-local measurements for the packaged Kylin NGCC instances.

Performance analysis. At the design level, Kylin's dominant costs are hash and XOF calls. Key generation builds the top hypertree layer, signing computes FORS material and one authentication path through each hypertree layer, and verification recomputes the FORS public key and verifies the hypertree signature. CS-WOTS$^+$ reduces the number of n-byte elements in each one-time signature, while Diverse Layer Heights keeps the signing hash-call proxy balanced even when h is not divisible by d.

Table 4: Design-level hash-call and signature-size metrics for the primary Kylin instances.

Instance	KeyGen hash calls	Sign hash calls	Verify hash calls	Signature bytes
Kylin-128	417279	2442880	7357	6688
Kylin-256	324095	3223168	12577	26784
Kylin-512	495615	6198272	42476	103808

Benchmark platform. The software implementation is written in ISO C and is compiled with GCC 13.3.0. The reference implementations keep their default `-O2` ISO C build with warning checks enabled. The optimized 64-bit PC implementations use per-directory builds centered on `-O3 -march=native`; `Kylin-128` additionally enables link-time optimization, omitted frame pointers, loop unrolling, and `-fno-plt`. The benchmark host is an x86-64 64-bit PC environment: AMD Ryzen 7 7800X3D 8-Core Processor, 8 physical cores and 16 hardware threads, approximately 4.192 GHz as reported by `/proc/cpuinfo`, 30 GiB RAM, Ubuntu 24.04.3 LTS on Linux 6.6.87.2-microsoft-standard-WSL2. The implementation uses the ICCS auxiliary hash/XOF interfaces `sm3hash`, `pseudohash`, and `pseudoXOF`; no external cryptographic library is required for the reported package benchmarks.

Software performance, transmission, and storage cost. The automated package measurements in Table 5 are produced from the current implementation tree by the package benchmark driver. The table reports the median of 9 measured host-local wall-clock timings on a 32-byte message after 3 warm-up iterations, and includes the public-key, secret-key, and signature sizes in bytes. Because these are repository-local wall-clock measurements, the absolute values should be interpreted as host-specific data points rather than cross-platform claims.

In the current rerun, the optimized implementations remain consistently faster than the reference ones. For `Kylin-128`, `Kylin-256`, and `Kylin-512`, the measured signing speedups are approximately 3.06×, 2.84×, and 3.16×, respectively; the key-generation speedups are approximately 3.85×, 4.28×, and 5.89×; and the verification speedups are approximately 2.58×, 2.75×, and 3.77×.

Table 5: Repository-local software performance and transmission/storage cost for the packaged Kylin NGCC instances. Times are milliseconds on a 32-byte message; sizes are bytes.

Instance	Impl.	KeyGen (ms)	Sign (ms)	Verify (ms)	Pk	Sk	Sig
Kylin-128	opt	75.058	592.035	1.989	32	64	6688
Kylin-128	ref	288.953	1813.298	5.141	32	64	6688
Kylin-256	opt	52.720	878.207	3.299	64	128	26784
Kylin-256	ref	225.417	2491.528	9.088	64	128	26784
Kylin-512	opt	171.051	4369.834	22.940	128	256	103808
Kylin-512	ref	1006.842	13801.995	86.494	128	256	103808
Kylin-128-q24	opt	344659	546652	0.147	32	64	3840
Kylin-128-q24	ref	853437	1072184	0.201	32	64	3840
Kylin-256-q24	opt	348092	1259275	0.385	64	128	14880
Kylin-256-q24	ref	859058	1875189	0.522	64	128	14880
Kylin-512-q24	opt	1092497	3311972	5.524	128	256	52672
Kylin-512-q24	ref	2731243	6623944	8.286	128	256	52672

12 Feature Statements

This section summarizes the main characteristics of Kylin in the categories requested by the call.

Innovativeness. Kylin introduces CS-WOTS$^+$, a constant-sum, size-optimal encoding for Winternitz-type one-time signatures, and combines it with Diverse Layer Heights in the hypertree parameter search. These two changes improve the signature-size and hash-call trade-off while preserving the conservative hash-based design line.

Simplicity. The scheme stays within the well-understood family of stateless hash-based signatures. Its core ingredients are tweakable hash functions, a one-time signature layer, FORS, and a hypertree, with no lattice, code, or multivariate algebraic structure.

Flexibility. The parameters (n, w, h, d, k, t) allow the construction to trade signature size, signing time, and verification time. The primary submission fixes the three `Kylin-128`, `Kylin-256`, and `Kylin-512` profiles, while the appendix records reference-only $q = 2^{24}$ profiles for lower signing-budget scenarios.

Compatibility. The packaged code follows the NGCC SIG API and routes hash/XOF calls through the ICCS auxiliary functions. The public workflow remains the conventional key-generation, signing, and verification interface. Because CS-WOTS$^+$ replaces WOTS$^+$, Kylin is not byte-for-byte compatible with the original SPHINCS$^+$ formats.

Extensibility. The construction is modular: replacing the hash/XOF instantiation or retuning the parameter set does not change the top-level signature workflow. The current submission, however, only claims digital-signature functionality.

Implementation performance advantages. The repository provides both ISO C reference implementations and optimized 64-bit PC implementations for the three primary instances. As shown in Table 5, the optimized implementation improves the measured key-generation, signing, and verification times over the reference implementation on the current 64-bit PC benchmark platform.

References

[1] Désiré André. Mémoire sur les combinaisons régulières et leurs applications. In *Annales scientifiques de l'École Normale Supérieure*, volume 5, pages 155–198, 1876.

[2] Jean-Philippe Aumasson and Guillaume Endignoux. Improving stateless hash-based signatures. In Nigel P. Smart, editor, *Topics in Cryptology – CT-RSA 2018*, volume 10808 of *Lecture Notes in Computer Science*, pages 219–242, San Francisco, CA, USA, April 16–20, 2018. Springer, Cham, Switzerland.

[3] Hacene Belbachir and Oussama Igueroufa. Congruence properties for bi s nomial coefficients and like extended ram and kummer theorems under suitable hypothesis. *Mediterranean Journal of Mathematics*, 17(1):1–14, 2020.

[4] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 368–397, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[5] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS$^+$ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 2129–2146, London, UK, November 11–15, 2019. ACM Press.

[6] Daniel Bleichenbacher and Ueli M. Maurer. Directed acyclic graphs, one-way functions and digital signatures. In Yvo Desmedt, editor, *Advances in Cryptology – CRYPTO'94*, volume 839 of *Lecture Notes in Computer Science*, pages 75–82, Santa Barbara, CA, USA, August 21–25, 1994. Springer Berlin Heidelberg, Germany.

[7] Jurjen N. Bos and David Chaum. Provably unforgeable signatures. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO'92*, volume 740 of *Lecture Notes in Computer Science*, pages 1–14, Santa Barbara, CA, USA, August 16–20, 1993. Springer Berlin Heidelberg, Germany.

[8] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for stateful hash-based signature schemes. *NIST Special Publication*, 800:208, 2020.

[9] Robert P Dilworth. A decomposition theorem for partially ordered sets. In *Classic Papers in Combinatorics*, pages 139–144. Springer, 2009.

[10] Leonhard Euler. De evolutione potestatis polynomialis cuiuscunque (1+ x+ x 2+ x 3+ x 4+ etc.) n. *Nova Acta Academiae Scientiarum Imperialis Petropolitanae*, pages 47–57, 1801.

[11] Oded Goldreich. Two remarks concerning the Goldwasser-Micali-Rivest signature scheme. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO'86*, volume 263 of *Lecture Notes in Computer Science*, pages 104–110, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.

[12] Oded Goldreich. *Foundations of Cryptography: Basic Applications*, volume 2. Cambridge University Press, Cambridge, UK, 2004.

[13] Shay Gueron and Nicky Mouha. SPHINCS-simpira: Fast stateless hash-based signatures with post-quantum security. Cryptology ePrint Archive, Report 2017/645, 2017.

[14] Douglas Zare (https://math.stackexchange.com/users/8345/douglas zare). How to express $(1 + x + x^2 + \cdots + x^m)^n$ as a power series? Mathematics Stack Exchange, 2011. URL:https://math.stackexchange.com/q/28861 (version: 2011-11-15).

[15] Andreas Hülsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. Xmss: extended merkle signature scheme. In *RFC 8391*. IRTF, 2018.

[16] Andreas Hülsing, Mikhail A. Kudinov, Eyal Ronen, and Eylon Yogev. SPHINCS+C: Compressing SPHINCS+ with (almost) no cost. In *2023 IEEE Symposium on Security and Privacy*, pages 1435–1453, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.

[17] Dmitry Khovratovich, Mikhail A. Kudinov, and Benedikt Wagner. At the top of the hypercube - better size-time tradeoffs for hash-based signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 93–123, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.

[18] Mikhail Kudinov, Andreas Hülsing, Eyal Ronen, and Eylon Yogev. Sphincs+c: Compressing sphincs+ with (almost) no cost. Cryptology ePrint Archive, Paper 2022/778, 2022. `https://eprint.iacr.org/2022/778`.

[19] Leslie Lamport. Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, October 1979.

[20] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-micali hash-based signatures. In *RFC 8554*. IRTF, 2019.

[21] Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO'89*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.

[22] National Institute of Standards and Technology. Module-lattice-based digital signature standard (fips 205). Technical report, U.S. Department of Commerce, 2024.

[23] National Institute of Standards and Technology. Recommendation for stateful hash-based signature schemes (initial public draft). Technical report, U.S. Department of Commerce, April 2026. Published April 13, 2026.

[24] Lucas Pandolfo Perin, Gustavo Zambonin, Ricardo Custódio, Lucia Moura, and Daniel Panario. Improved constant-sum encodings for hash-based signatures. *Journal of Cryptographic Engineering*, pages 1–23, 2021.

[25] Zhen Qin and Siwei Sun. Extending the SPHINCS+ framework: Varying the tree heights and chain lengths. Cryptology ePrint Archive, Report 2025/2236, 2025.

[26] Leonid Reyzin and Natan Reyzin. Better than biba: Short one-time signatures with fast signing and verifying. Cryptology ePrint Archive, Report 2002/014, 2002.

[27] Emanuel Sperner. Ein satz über untermengen einer endlichen menge. *Mathematische Zeitschrift*, 27(1):544–548, 1928.

[28] Serge Vaudenay. One-time identification with low memory. In *Eurocode'92*, pages 217–228. Springer, 1993.

[29] S Ole Warnaar. The andrews–gordon identities and q-multinomial coefficients. *Communications in mathematical physics*, 184(1):203–232, 1997.

[30] Kaiyi Zhang, Hongrui Cui, and Yu Yu. Revisiting the constant-sum winternitz one-time signature with applications to sphincs+ and xmss. In *Advances in Cryptology–CRYPTO 2023: 43th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–24, 2023, Proceedings, to appear*, pages xxx–xxx. Springer, 2023.

A Parameter Evaluation Sage Script

The script below evaluates parameter candidates with per-layer hypertree heights $(h_0, \ldots, h_{d-1})$ satisfying $\sum_{i=0}^{d-1} h_i = h$. It uses h_0 for the message-digest index split and uses $\sum_{i=0}^{d-1} 2^{h_i}$ in the HT hash-call proxy. The currently frozen rows include the packaged $q = 2^{64}$ tuples `Kylin-128/256/512` and the companion $q = 2^{24}$ tuples `Kylin-128-q24/256-q24/512-q24`.

```
#!/usr/bin/env python3
"""Reference parameter-evaluation script with per-layer HT heights h_vec.

This appendix script mirrors the frozen NGCC parameter set used by the current
repository package:
  - q = 2^64 signatures per key pair
  - each family has its own fixed tuple (h, d, h_vec, a, k, w, len)
  - digest split uses h_0 (lowest layer)
  - HT signing proxy uses sum_i 2^{h_i}
"""

from __future__ import annotations

from functools import lru_cache

import mpmath as mp

mp.mp.dps = 80

Q_SIGS = 2**64
MAX_LAYER_HEIGHT = 32

FAMILIES = (
    {
        "name": "Kylin-128",
        "n": 16,
        "target_classical": 128,
        "ngcc_quantum": 80,
        "h": 65,
        "d": 9,
        "h_vec": (7, 7, 7, 7, 7, 7, 7, 8, 8),
        "a": 13,
        "k": 11,
```

```
        "w": 73,
        "len": 22,
    },
    {
        "name": "Kylin-256",
        "n": 32,
        "target_classical": 256,
        "ngcc_quantum": 128,
        "h": 68,
        "d": 10,
        "h_vec": (6, 6, 7, 7, 7, 7, 7, 7, 7, 7),
        "a": 13,
        "k": 22,
        "w": 54,
        "len": 46,
    },
    {
        "name": "Kylin-512",
        "n": 64,
        "target_classical": 512,
        "ngcc_quantum": 256,
        "h": 70,
        "d": 4,
        "h_vec": (17, 17, 18, 18),
        "a": 20,
        "k": 27,
        "w": 64,
        "len": 87,
    },
)


def balanced_profile(h: int, d: int) -> tuple[int, ...]:
    base = h // d
    rem = h % d
    return tuple(base + (1 if i < rem else 0) for i in range(d))


def shift_height(profile: tuple[int, ...], src: int, dst: int, amount: int) -> tuple[int, ...] | None:
    if amount <= 0:
        return profile
    if src == dst:
        return profile
    p = list(profile)
    if p[src] - amount < 1:
        return None
    p[src] -= amount
    p[dst] += amount
    return tuple(p)


def profile_ok(h: int, h_vec: tuple[int, ...]) -> bool:
    if not h_vec:
        return False
    if any(v < 1 for v in h_vec):
        return False
    if sum(h_vec) != h:
        return False
    if max(h_vec) > MAX_LAYER_HEIGHT:
        return False
    # tree_idx fits in 64 bits for the current implementation split.
    if h - h_vec[0] > 64:
        return False
    return True


def generate_profiles(h: int, d: int) -> list[tuple[int, ...]]:
    core = balanced_profile(h, d)
    profiles: set[tuple[int, ...]] = {core}

    top = d - 1
    mid = d // 2
    for candidate in (
        shift_height(core, top, 0, 1),
        shift_height(core, top, 0, 2),
        shift_height(core, 0, top, 1),
        shift_height(core, 0, top, 2),
        shift_height(core, mid, 0, 1),
        shift_height(core, mid, top, 1),
        shift_height(core, top, mid, 1),
        shift_height(core, 0, mid, 1),
    ):
        if candidate is not None and profile_ok(h, candidate):
            profiles.add(candidate)

    return sorted(profiles)


@lru_cache(maxsize=None)
def const_sum_count(length: int, w: int) -> int:
    total = length * (w - 1) // 2
    dp = [0] * (total + 1)
    dp[0] = 1
    for _ in range(length):
        nxt = [0] * (total + 1)
        window = 0
        for idx in range(total + 1):
            window += dp[idx]
            if idx - w >= 0:
                window -= dp[idx - w]
            nxt[idx] = window
        dp = nxt
    return dp[total]


@lru_cache(maxsize=None)
def wotscs_length(message_bits: int, w: int) -> int:
    for length in range(2, 700):
        if mp.log(const_sum_count(length, w), 2) > message_bits:
            return length
    raise RuntimeError(f"failed to find WOTSCS length for {message_bits=} {w=}")


@lru_cache(maxsize=None)
def qhit_distribution(h: int, limit: int = 220) -> tuple[mp.mpf, ...]:
    p = mp.mpf(2) ** (-h)
```

```
    q = mp.mpf(Q_SIGS)
    cur = mp.e ** (q * mp.log1p(-p))
    probs = [cur]
    for r in range(limit):
        cur *= ((q - r) / (r + 1)) * (p / (1 - p))
        probs.append(cur)
    return tuple(probs)


@lru_cache(maxsize=None)
def classical_security(n: int, h: int, a: int, k: int) -> mp.mpf:
    total = mp.mpf(2) ** (-8 * n)
    tree_size = mp.mpf(2) ** a
    for r, qprob in enumerate(qhit_distribution(h)):
        total += qprob * (1 - (1 - 1 / tree_size) ** r) ** k
    return -mp.log(total, 2)


def minimal_k(target_bits: int, n: int, h: int, a: int) -> int:
    upper = 240 if target_bits < 512 else 420
    lower = 10 if target_bits < 512 else 30
    if classical_security(n, h, a, upper) < target_bits:
        raise RuntimeError(f"no feasible k found for n={n}, h={h}, a={a}")

    lo, hi = lower, upper
    while lo < hi:
        mid = (lo + hi) // 2
        if classical_security(n, h, a, mid) >= target_bits:
            hi = mid
        else:
            lo = mid + 1
    return lo


def signature_bytes(n: int, h: int, d: int, a: int, k: int, length: int) -> int:
    return ((a + 1) * k + h + length * d + 1) * n


def hash_calls(h_vec: tuple[int, ...], a: int, k: int, w: int, length: int) -> int:
    return k * (2 ** (a + 1)) + sum((2**layer_h) * (length * w + 1) for layer_h in h_vec)


def digest_bytes(k: int, a: int, h: int, h0: int) -> int:
    tree_bits = h - h0
    return ((k * a + 7) // 8) + ((tree_bits + 7) // 8) + ((h0 + 7) // 8)


def choose_profile(h: int, d: int, a: int, k: int, w: int, length: int) -> tuple[int, ...]:
    profiles = generate_profiles(h, d)
    return min(profiles, key=lambda vec: (hash_calls(vec, a, k, w, length), vec))


def evaluate_family(family: dict[str, int | str | tuple[int, ...]]) -> dict[str, int | float | str]:
    h = int(family["h"])
    d = int(family["d"])
    h_vec = tuple(int(x) for x in family["h_vec"])  # type: ignore[arg-type]
    n = int(family["n"])
    a = int(family["a"])
    k = int(family["k"])
    w = int(family["w"])
    length = int(family["len"])

    return {
        "name": str(family["name"]),
        "n": n,
        "h": h,
        "d": d,
        "h_vec": h_vec,
        "h0": h_vec[0],
        "a": a,
        "k": k,
        "w": w,
        "len": length,
        "classical": float(classical_security(n, h, a, k)),
        "quantum_formula": float(classical_security(n, h, a, k) / 2),
        "ngcc_quantum": int(family["ngcc_quantum"]),
        "pk": 2 * n,
        "sk": 4 * n,
        "sig": signature_bytes(n, h, d, a, k, length),
        "digest_bytes": digest_bytes(k, a, h, h_vec[0]),
        "hashcalls": hash_calls(h_vec, a, k, w, length),
    }


def verify_frozen_parameters(rows: list[dict[str, int | float | str]]) -> None:
    expected: dict[str, tuple[int, int, tuple[int, ...], int, int, int, int]] = {
        "Kylin-128": (65, 9, (7, 7, 7, 7, 7, 7, 7, 8, 8), 13, 11, 73, 22),
        "Kylin-256": (68, 10, (6, 6, 7, 7, 7, 7, 7, 7, 7, 7), 13, 22, 54, 46),
        "Kylin-512": (66, 11, (6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6), 13, 49, 97, 79),
    }
    for row in rows:
        name = str(row["name"])
        if name not in expected:
            raise AssertionError(f"unexpected family: {name}")

        exp_h, exp_d, exp_h_vec, exp_a, exp_k, exp_w, exp_len = expected[name]
        h = int(row["h"])
        d = int(row["d"])
        length = int(row["len"])
        a = int(row["a"])
        k = int(row["k"])
        w = int(row["w"])
        h_vec = tuple(row["h_vec"])  # type: ignore[arg-type]

        if (h, d, h_vec, a, k, w, length) != (exp_h, exp_d, exp_h_vec, exp_a, exp_k, exp_w, exp_len):
            raise AssertionError(f"parameter drift for {name}: {(h, d, h_vec, a, k, w, length)}")

        recomputed_hashcalls = hash_calls(h_vec, a, k, w, length)
        if int(row["hashcalls"]) != recomputed_hashcalls:
            raise AssertionError(f"hashcalls mismatch for {row['name']}")


def main() -> None:
    rows = [evaluate_family(family) for family in FAMILIES]
    verify_frozen_parameters(rows)
```

```
    for row in rows:
        print(
            f"{row['name']}: h={row['h']}, d={row['d']}, h_vec={row['h_vec']}, "
            f"a={row['a']}, k={row['k']}, w={row['w']}, len={row['len']}, "
            f"classical={row['classical']:.1f}, ngcc_quantum={row['ngcc_quantum']}, "
            f"formula_quantum={row['quantum_formula']:.1f}, "
            f"pk={row['pk']}, sk={row['sk']}, sig={row['sig']}, "
            f"digest_bytes={row['digest_bytes']}, hashcalls={row['hashcalls']}"
        )


if __name__ == "__main__":
    main()
```

B $q = 2^{24}$ Companion Parameter Profiles

This appendix summarizes the frozen companion $q = 2^{24}$ profiles that are kept as additional reference implementations. The motivation is practical: some application settings remain dissatisfied with large signature sizes and verification-time behavior under very large per-key signing budgets. In addition, NIST's April 13, 2026 initial public draft SP 800-230 highlights stateful hash-based signatures with explicit per-key usage bounds, which further motivates keeping explicit companion rows for limited-signature operating points (including 2^{24}-style limits) [23].

In this repository, the $q = 2^{24}$ rows are selected from the same constrained search workflow and then frozen as reference-only companions to the packaged NGCC $q = 2^{64}$ families.

Table 6: Frozen companion $q = 2^{24}$ Kylin parameter profiles (reference-only additional implementations).

Family	NGCC lvl	n	h	d	$\log t$	k	w	l	bitsec	pk	sk	sig
Kylin-128-q24	128/80	16	22	1	24	6	4	67	127	32	64	3840
Kylin-256-q24	256/128	32	21	1	25	12	4	131	255	64	128	14880
Kylin-512-q24	512/256	64	57	3	20	24	64	87	511	128	256	52672

The benchmark rows in Table 7 mirror the $q = 2^{24}$ entries recorded in `package_bench.csv`.

Table 7: Package-benchmark results for companion $q = 2^{24}$ Kylin implementations. Times are milliseconds on a 32-byte message; signature sizes are bytes.

Family	impl	keygen (ms)	sign (ms)	verify (ms)	sig (B)
Kylin-128-q24	ref	853,437	1,072,184	0.201	3,840
Kylin-128-q24	opt	344,659	546,652	0.147	3,840
Kylin-256-q24	ref	859,058	1,875,189	0.522	14,880
Kylin-256-q24	opt	348,092	1,259,275	0.385	14,880
Kylin-512-q24	ref	2,731,243	6,623,944	8.286	52,672
Kylin-512-q24	opt	1,092,497	3,311,972	5.524	52,672

The optimized implementations apply macro-unrolled SM3 compression (64 explicit rounds, no branches), SSE2/AVX2 hardware dispatch for message-word loading, a malloc-free `pseudoXOF` fast path for byte-aligned input, and 4-way SSE2 WOTS chain acceleration (`thash_wots_chain_4x`) adapted for $n = 16/32/64$. Both the sign and verify paths batch 4 WOTS chains per SIMD operation. The KAT output of every optimized variant matches the corresponding reference byte-for-byte (SHA-256 verified).